DOCUMENT DE TRANSMISSION — CONFIDENTIEL

Cahier des charges

Reprise du projet « Pointage & Rémunération » et migration de Firebase vers un backend PostgreSQL

Application de gestion des salariés, pointage GPS, rémunération, matériel et messagerie

Projet	pointage-app (dépôt « gestion-salaries », package `pointage-app` v1.0.0)
Identifiant Android	`mg.fortico.pointage` (Capacitor)
Projet Firebase actuel	`pointage-c41b0` (Auth + Firestore + Analytics)
Version du document	1.0 — 1er octobre 2026
Base de l'analyse	Code source réel du dépôt (branche `main`, dernier commit « mise a jour ok 23 »). Aucune fonctionnalité n'a été inventée : les points non implémentés ou ambigus sont signalés comme tels.

Sommaire

1. Contexte, objectifs et périmètre
2. Vue d'ensemble de l'application actuelle
3. Utilisateurs, rôles et permissions
4. Description détaillée des pages
5. Parcours utilisateurs et processus métier
6. Règles métier à conserver (calculs et contrôles)
7. Données actuelles dans Firebase (Firestore et Auth)
8. Règles de sécurité et contrôles d'accès actuels
9. Flux de données UI ⇄ Firebase (opérations par page)
10. Notifications, recherches, filtres, validations
11. Dépendances et services externes
12. Points d'attention et anomalies constatées dans le code actuel
13. Architecture cible : backend + PostgreSQL
14. Schéma PostgreSQL détaillé
15. Authentification et autorisation
16. API REST : endpoints principaux
17. Temps réel (messagerie, matériel)
18. Validations et gestion des erreurs
19. Communication du frontend avec le nouveau backend
20. Éléments Firebase à supprimer ou remplacer
21. Migration des données, plan de travail et recette
22. Points importants à conserver lors de la migration
23. Annexes : glossaire et statuts

1. Contexte, objectifs et périmètre

1.1 Objet de l'application

L'application permet à une entreprise (Madagascar : monnaie **Ariary (Ar)**, fuseau UTC+3, interface entièrement en français) de :

- gérer ses **salariés** (fiches, comptes de connexion, affectation à des départements) ;
- faire **pointer** les salariés (entrée / sortie) depuis leur téléphone avec **vérification GPS** du lieu de travail (géofencing) ;
- calculer automatiquement la **rémunération journalière** à partir du temps travaillé et tenir un **solde cumulé** ;
- enregistrer les **paiements en espèces** faits aux salariés et suivre le **reste à payer** ;
- gérer le **matériel** prêté aux salariés (inventaire, demandes d'emprunt, retours) ;
- offrir une **messagerie interne** (groupes, annonces, messages directs admin ↔ salarié) ;
- fournir à l'administrateur un **tableau de bord**, une **carte** des pointages, des **rapports** (export CSV) et des **paramètres**.

1.2 Objectif de la mission de reprise

Supprimer entièrement Firebase (Authentication, Cloud Firestore, règles de sécurité, Analytics, SDK `firebase`) et le remplacer par un **backend propre** exposant une **API** et s'appuyant sur **PostgreSQL**. L'interface (React) et le comportement fonctionnel doivent rester **identiques** du point de vue de l'utilisateur ; seule la couche de données et d'authentification change.

1.3 Périmètre

Inclus	Nouveau backend (API + PostgreSQL), authentification propre, migration des données Firestore/Auth, adaptation de la couche `src/services` et des contextes React, remplacement du temps réel Firestore (`onSnapshot`), suppression de Firebase, tests et recette, déploiement.
Exclu (sauf demande)	Refonte graphique, nouvelles fonctionnalités métier, push notifications serveur (l'application n'utilise que des notifications locales Android, voir §10), export PDF/Excel, congés/absences, mode hors ligne (listés « non implémentés » dans le README d'origine).

> **Conventions de ce document.** « Salarié » = rôle `employee`, « Admin » = rôle `admin`. Les noms de fichiers sont relatifs à `src/`. Les noms de collections Firestore sont donnés tels quels (`employees`, `attendance`…). Les propositions de la partie « Architecture cible » (§13 et suivants) sont des **recommandations** : le développeur reste libre de les ajuster tant que les règles métier des §6 et §22 sont respectées.

2. Vue d'ensemble de l'application actuelle

2.1 Architecture technique actuelle

Application **100 % frontend** (SPA) qui parle **directement** à Firebase depuis le navigateur. Il n'existe **aucun serveur applicatif** ni Cloud Function : toute la logique (calcul de salaire, contrôle GPS, cumul du solde, agrégations) s'exécute dans le navigateur ; la seule protection côté serveur est le fichier `firestore.rules`.

Couche	Technologie / détail
Frontend	React 18 + TypeScript, Vite 5, Tailwind CSS 3 (mode sombre par classe), React Router 6, lucide-react (icônes), Leaflet / react-leaflet (cartes), Recharts (installé ; non utilisé dans les pages lues).
Mobile	Capacitor 8 (Android). `capacitor.config.ts` : `appId mg.fortico.pointage`, la WebView charge l'URL distante `https://suivie-fortico.vercel.app/login` (pas de bundle embarqué). Plugins : `@capacitor/geolocation`, `@capacitor/local-notifications`.
Hébergement web	Vercel (`vercel.json` : toutes les routes réécrites vers `/index.html`, SPA).
Backend actuel	Firebase : Authentication (email + mot de passe), Cloud Firestore (base NoSQL), Analytics (initialisé dans `services/firebase.ts`). Projet `pointage-c41b0`.
Configuration	La configuration Firebase (clé API, projectId…) est **écrite en dur** dans `services/firebase.ts` (le commentaire parle de `.env` mais il n'est pas utilisé). Aucun fichier `.env` dans le dépôt.

2.2 Organisation du code (`src/`)

Dossier / fichier	Rôle
`main.tsx`	Point d'entrée : `BrowserRouter` → `ThemeProvider` → `AuthProvider` → `ConfirmProvider` → `App`.
`routes/AppRoutes.tsx`	Table des routes (voir §4). `routes/ProtectedRoute.tsx` protège par rôle.
`contexts/`	`AuthContext` (session + profil applicatif), `ChatContext` (conversations, non-lus, notifications locales), `ConfirmContext` (fenêtre de confirmation, remplace `window.confirm`), `ThemeContext` (clair/sombre, `localStorage["theme"]`).
`services/`	**Seule couche qui touche Firebase** : `firebase.ts, auth.ts, employees.ts, workplaces.ts, attendance.ts, salary.ts, payments.ts, settings.ts, equipment.ts, chat.ts, audit.ts, reset.ts` ; plus `notify.ts` (notifications locales, sans Firebase). C'est ici que se concentre le travail de migration.
`utils/`	`salaryCalculator.ts` (règle de paie), `distance.ts` (Haversine), `geolocation.ts` (GPS + Nominatim), `date.ts` (clé `YYYY-MM-DD` locale), `format.ts`.
`pages/`	`auth/Login`, `admin/*` (10 pages), `employee/*` (6 pages).
`layouts/`	`AdminLayout` (barre latérale + en-tête), `EmployeeLayout` (barre d'onglets en bas, style mobile).
`components/`	Composants UI (Card, Badge, Toast, Dropdown, Sidebar, Header, EmptyState, PasswordInput, ThemeToggle), cartes (`LocationPicker, MapLayers, mapIcons`), messagerie (`chat/ChatPage, chat/ChatConversation`).

2.3 Utilisation de l'heure et des dates

- Les « jours » de pointage sont des clés `YYYY-MM-DD` calculées sur l'**heure locale du navigateur** (`utils/date.ts`, volontairement sans `toISOString()` pour éviter le décalage UTC+3). Le backend devra

définir le jour de travail dans le fuseau **Indian/Antananarivo** (UTC+3, sans heure d'été).
- Les heures d'entrée/sortie utilisent aujourd'hui `serverTimestamp()` pour l'écriture, mais le **calcul de la durée** utilise l'horloge du navigateur à l'instant de la sortie (`new Date()`) : le backend devra tout calculer avec sa propre horloge.

3. Utilisateurs, rôles et permissions

3.1 Les deux rôles

Rôle	Description	Connexion
admin	Gestionnaire de l'entreprise : accès complet à toutes les données et à toutes les actions. Plusieurs admins possibles. L'admin n'est lié à aucune fiche salarié.	Email + mot de passe
employee	Salarié lié à **une fiche** `employees/{EMP-xxx}`. Ne voit que ses propres données (pointages, rémunération, prêts) plus les données partagées (lieux, paramètres, matériel, groupes de discussion). Un salarié **désactivé** ou dont la fiche est supprimée est considéré comme sans accès : renvoyé vers `/login`.	Identifiant (ex. `EMP-001`) ou email + mot de passe / code PIN (6 caractères min.)

Le rôle est stocké dans le document `users/{uid}` (`role`, et `employeeId` pour un salarié). Résolution : `services/auth.ts → getAppUser()`. Un utilisateur Firebase sans document `users` n'a aucun accès.

3.2 Matrice des permissions (comportement actuel)

Ressource / action	Admin	Salarié	Remarques
Se connecter, se déconnecter	oui	oui	Salarié actif uniquement.
Créer un compte admin (écran de connexion)	—	—	Public mais protégé par un **code d'invitation** (`admin_invite/main.code`) vérifié par les règles Firestore (§8).
Fiches salariés : lire / créer / modifier / (dés)activer / supprimer	oui	lecture de la **sienne**	Création = fiche + compte de connexion.
Changer le mot de passe d'un salarié	oui*	non	*Aujourd'hui possible seulement en connaissant l'ancien mot de passe, ou par email de réinitialisation (limite du SDK client).
Lieux de travail (départements)	CRUD	lecture	
Paramètres de rémunération	lecture + écriture	lecture	Document unique `settings/general`.
Pointage : entrée / sortie (GPS vérifié)	—	oui (pour lui-même)	Un salarié ne peut ni modifier ni supprimer un pointage terminé.
Pointage à distance (sans GPS) : démarrer / arrêter	oui	non	Marqué `source = ADMIN`.
Lire les pointages	tous	les siens	Aucune UI pour modifier/supprimer un pointage (même si les règles l'autorisent à l'admin).
Rémunérations (`salary_records`) : lire	toutes	les siennes	
Paiements : créer / lire	oui / tous	non / les siens	Aucune UI de suppression/modification.
Matériel : inventaire	CRUD	lecture	
Prêts : demander / annuler / signaler retour	oui (règles)	pour lui-même	Voir cycle §5.5.
Prêts : approuver / refuser / confirmer retour	oui	non	

Messagerie : créer groupe / message direct, vider, supprimer conversation, supprimer un message	oui	non	Seul l'admin supprime les messages.
Messagerie : rejoindre / quitter un groupe, écrire, réagir	voit et écrit partout	oui (membre)	Groupe « annonce » : lecture seule pour les salariés.
Tableau de bord, carte, rapports, réinitialisation des données	oui	non	Réinitialisation protégée par re-saisie du mot de passe admin.
Journal d'audit	lecture	écriture seulement	Aucune page ne l'affiche aujourd'hui.

3.3 Contrôle d'accès côté interface

- `ProtectedRoute allow="admin|employee"` : si le chargement est en cours → « Chargement… » ; si non connecté ou profil introuvable/désactivé → `/login` ; si mauvais rôle → redirection vers l'espace de son propre rôle (`/admin` ou `/employe`).
- Après connexion, `Login` redirige automatiquement : admin → `/admin`, salarié → `/employe`.
- Ces contrôles sont **cosmétiques** : la vraie sécurité devra être appliquée par le backend sur chaque endpoint (§15).

4. Description détaillée des pages

4.1 Table des routes

URL	Page (fichier)	Accès
`/login` (et `/`, `*`)	Login (`pages/auth/Login.tsx`)	public
`/admin`	Dashboard	admin
`/admin/salaries-personnel`	Salariés (`Employees.tsx`)	admin
`/admin/presences`	Présences (`Attendance.tsx`)	admin
`/admin/carte`	Carte des pointages (`MapView.tsx`)	admin
`/admin/remunerations`	Rémunérations (`Salaries.tsx`)	admin
`/admin/discussions`	Discussions (`Chat.tsx` → `ChatPage`)	admin
`/admin/materiel`	Matériel (`Equipment.tsx`)	admin
`/admin/rapports`	Rapports (`Reports.tsx`)	admin
`/admin/lieux`	Lieux de travail (`Workplaces.tsx`)	admin
`/admin/parametres`	Paramètres (`Settings.tsx`)	admin
`/employe`	Accueil / pointage (`employee/Home.tsx`)	salarié
`/employe/historique`	Historique (`History.tsx`)	salarié
`/employe/remuneration`	Salaire (`Salary.tsx`)	salarié
`/employe/discussions`	Discussions (`Chat.tsx`)	salarié
`/employe/materiel`	Matériel (`Equipment.tsx`)	salarié
`/employe/profil`	Profil (`Profile.tsx`)	salarié

4.2 Page de connexion — `/login`

Rôle : unique point d'entrée. Écran scindé : bandeau marketing (masqué sur mobile) + formulaire.

- **Mode « connexion »** : champs *Identifiant / Email* et *Mot de passe / PIN* (avec bouton afficher/masquer), case « Se souvenir de mon identifiant » (`localStorage["rememberedId"]`). Si l'identifiant ne contient pas de « @ », il est traité comme code salarié et résolu en email de connexion via le document `employees/{id}.loginEmail` ; puis `signInWithEmailAndPassword`. Toute erreur affiche le même message : « Identifiant ou mot de passe incorrect. ».
- **Mode « création d'un compte admin »** (lien « Créer un compte administrateur ») : email, mot de passe (≥ 6), confirmation, **code d'invitation administrateur**. Crée le compte Auth sur une instance Firebase secondaire, écrit `users/{uid} = {role:"admin", inviteCode}` (accepté seulement si le code correspond à

`admin_invite/main.code`), puis connecte automatiquement. Erreurs gérées : email déjà utilisé, email invalide, mot de passe faible, code incorrect (`permission-denied`).
- Redirection automatique si déjà connecté (admin → `/admin`, salarié → `/employe`).

4.3 Espace administrateur (layout `AdminLayout`)

Barre latérale (tiroir sur mobile) avec 10 entrées, en-tête affichant l'email de l'admin et le bouton « Déconnexion » (avec confirmation). Deux **badges** temps réel : nombre de messages non lus (Discussions) et nombre de demandes de matériel à traiter (Matériel). Le `ChatProvider` est monté pour tout l'espace, donc la messagerie est « vivante » sur toutes les pages.

A. Dashboard — `/admin`

Rôle : photographie de la journée. **Données lues :** tous les salariés, les pointages du jour, les paramètres, et pour chaque salarié actif son solde gagné et ses paiements.

Bloc	Calcul actuel (côté navigateur)
Salariés actifs	nombre de salariés avec `active = true`.
Présents	nombre de salariés **distincts** ayant au moins un pointage aujourd'hui (même encore en cours).
Absents	salariés actifs sans aucun pointage aujourd'hui (liste de badges).
Heures cumulées	somme de `workedMinutes` des pointages du jour / 60 (une session en cours compte 0 tant qu'elle n'est pas terminée).
Paie du jour estimée	somme de `dailySalary` des pointages du jour.
Cash restant à payer (global)	Σ sur les salariés actifs de (solde gagné − total payé). Requêtes N+1 (2 par salarié).
Heures travaillées aujourd'hui (top 5)	minutes cumulées par salarié (plusieurs sessions additionnées), tri décroissant, 5 premiers.
En poste actuellement	pointages `EN_COURS` avec l'heure d'entrée.
Retards	Pour la **première entrée** de chaque salarié : retard = heure d'entrée − (`normalEntryTime` + `toleranceMinutes`) ; affiché si > 0 min, trié par retard décroissant.

Backend cible : un endpoint agrégé `GET /dashboard/today` supprimera le N+1.

B. Salariés — `/admin/salaries-personnel`

Rôle : gestion complète des fiches et des comptes de connexion des salariés.

- **Tableau** : nom + identifiant, poste + départements, salaire journalier, statut (Actif/Inactif) et actions : *Voir*, *Modifier*, *Activer/Désactiver*, *Supprimer* (avec confirmation).
- **Modale « Créer / Modifier »** (le même formulaire) : identifiant (EMP-001, non modifiable en édition), prénom*, nom*, email pro, email de connexion Firebase*, téléphone, poste, date d'embauche, adresse, salaire de base (Ar pour une journée de 8 h, valeur par défaut = `settings.dailySalary`), **départements** (cases à cocher, au moins 1 obligatoire), et en création un **mot de passe / PIN** (≥ 6 caractères). (* = obligatoire.)
- **Section « Modifier le mot de passe du compte »** (édition uniquement) : « mot de passe actuel » + « nouveau mot de passe » → connexion temporaire au compte du salarié pour appeler `updatePassword` ; ou lien « Envoyer un email de réinitialisation » (`sendPasswordResetEmail` vers l'email de connexion).
- **Modale « Fiche salarié & montants »** (bouton œil) : solde cumulé total (`getCumulativeBalance`), poste, statut, salaire de base, emails, téléphone, date d'embauche, adresse, départements autorisés.
- Création : `createEmployee` = création du compte Auth (instance secondaire pour ne pas déconnecter l'admin) + document `users/{uid}` (`role employee`) + document `employees/{employeeId}` (l'ID du document est le code salarié).

- Suppression : supprime `employees/{id}` et `users/{uid}` ; le compte Auth reste orphelin (limite du SDK client) ; l'historique (pointages, rémunérations, paiements) n'est **pas** supprimé.

C. Présences — `/admin/presences`

- Sélecteur de date (défaut : aujourd'hui) ; tableau des pointages du jour : salarié, département, entrée, sortie, temps travaillé, salaire, GPS (distance en m, ou « Admin » si pointé à distance), statut (En cours / Terminée).
- **« Démarrer un pointage »** (aujourd'hui seulement) : choisir un salarié actif *sans session ouverte* et un département (par défaut ceux du salarié) → `adminClockIn` : crée un pointage `EN_COURS` sans GPS (`entrySource = ADMIN`, `entryBy = uid admin`), journalise `ATTENDANCE_CREATED`.
- **« Arrêter »** (sur une ligne `EN_COURS`, aujourd'hui seulement, après confirmation) : `adminClockOut` clôture avec la même logique de calcul que la sortie du salarié (salaire de base individuel + durée normale globale), `exitSource = ADMIN`, journalise `ATTENDANCE_UPDATED`.
- Aucune action pour corriger/supprimer un pointage passé.

D. Carte — `/admin/carte`

Carte Leaflet (fonds Plan OSM / Satellite Esri / Hybride) centrée sur Toamasina par défaut, recadrée automatiquement sur les marqueurs (ou les lieux de travail s'il n'y a pas de pointage). Filtres : date, salarié, type (Entrée/Sortie), case « Lieux de travail ». Affiche les cercles (rayon) et marqueurs des lieux, et un marqueur par entrée/sortie **ayant une position GPS** (les pointages admin n'en ont pas). Popup : nom, code, type, département, date, heure, latitude/longitude, distance au département, adresse (entrée).

E. Rémunérations — `/admin/remunerations`

- Trois cartes : Total gagné, Total déjà payé, Total restant à payer. Une ligne par salarié : Gagné / Payé / **Cash restant** (= gagné − payé).
- **« Payer »** (désactivé si reste ≤ 0) : formulaire montant (≤ reste à payer) + note facultative → `createPayment` (date = aujourd'hui) + audit `PAYMENT_CREATED`. Message de succès « X Ar payé à Prénom Nom ».
- **« Historique »** : liste des paiements du salarié (date, note, montant).

F. Discussions — `/admin/discussions` (voir §4.5)

G. Matériel — `/admin/materiel`

Trois onglets : **Demandes** (compteur ; demandes d'emprunt `DEMANDE` et retours à confirmer `RETOUR_DEMANDE`) avec actions *Confirmer le prêt*, *Refuser* (motif facultatif), *Confirmer le retour* ; **Inventaire** (ajouter / modifier / mettre hors service ↔ disponible / supprimer, lien vers l'historique d'un matériel) ; **Historique** (tous les prêts, filtres par matériel et par statut). Données en **temps réel** (`onSnapshot`). Détails du cycle : §5.5.

H. Rapports — `/admin/rapports`

Choix d'une période (défaut : du 1er du mois en cours à aujourd'hui) ; bouton « Générer » (aussi lancé à l'ouverture). Pour chaque salarié : **jours travaillés** (dates distinctes, donc plusieurs sessions le même jour = 1 jour), **heures** (Σ minutes / 60, 1 décimale), **salaire total** (Σ `dailySalary`), trié par salaire décroissant, avec total général. **Export CSV** (colonnes : Salarié, Identifiant, Jours travaillés, Heures travaillées, Salaire total ; fichier `rapport_{début}_{fin}.csv`). Pas d'export PDF/Excel.

I. Lieux de travail — `/admin/lieux`

Liste et formulaire (créer/modifier) de **lieux = départements** : nom, rayon autorisé (m, > 0, défaut 100), latitude/longitude (saisie ou clic/recherche sur carte via `LocationPicker` + recherche d'adresse Nominatim), interrupteur « Activer le géofencing ». Actions : modifier, activer/désactiver le géofencing en un clic, supprimer

(confirmation ; les salariés affectés ne pourront plus l'utiliser). Les anciennes données aux coordonnées invalides sont signalées (conversion en nombre, `NaN` si inutilisable).

J. Paramètres — `/admin/parametres`

- **Apparence** : thème clair/sombre (préférence locale du navigateur).
- **Rémunération** (document `settings/general`) : salaire pour 8 h (Ar) — valeur par défaut proposée aux nouveaux salariés —, heures normales par jour (stockées en minutes), heure d'entrée normale, tolérance de retard (min). Affiche le taux horaire calculé et le plafond journalier.
- **Zone dangereuse** : « Réinitialiser les données » : redemande le mot de passe admin, double confirmation, puis supprime : discussions (messages + accusés), pointages, rémunérations, paiements, prêts, matériel, **salariés**, et les documents `users` non-admin. Conservés : comptes admin, paramètres, lieux de travail, journal d'audit.

4.4 Espace salarié (layout `EmployeeLayout`, interface mobile)

En-tête (code salarié + déconnexion) et **barre d'onglets fixe en bas** : Accueil, Discussions (badge non-lus), Matériel, Salaire, et un menu « ⋯ » (Historique, Profil, Actualiser = rechargement de la page). La messagerie occupe tout l'écran.

K. Accueil / pointage — `/employe`

- Carte « solde global » (solde cumulé gagné) et date du jour.
- **Pas de pointage** : bouton « POINTAGE ENTRÉE » → fenêtre de choix du département (parmi ceux **affectés au salarié** ; si aucun : « Aucun département ne vous est affecté. Contactez votre administrateur. ») → capture GPS (`useGeolocation`, + adresse par géocodage inverse Nominatim, facultatif) → `clockIn`.
- **Session en cours** : heure d'entrée, durée écoulée (rafraîchie toutes les 30 s), **salaire estimé en direct**, origine (« Pointage démarré par l'administrateur » ou adresse/« Position GPS validée »), bouton « POINTAGE SORTIE » → nouvelle capture GPS → `clockOut` sur le **même département** que l'entrée.
- **Session terminée** : récapitulatif (entrée, sortie, durée, salaire, total du jour sur n sessions) et bouton « NOUVEAU POINTAGE ENTRÉE » (plusieurs sessions par jour autorisées, jamais deux ouvertes simultanément).
- Refus GPS : « Pointage refusé — Vous êtes actuellement à X mètres du département sélectionné. Vous devez être dans un rayon de R mètres… ». Succès : « Entrée/Sortie enregistrée — Département : … · Distance : N m ».

L. Historique — `/employe/historique`

60 derniers pointages du salarié, du plus récent au plus ancien : date, durée, heure d'entrée et de sortie (ou « En cours »).

M. Salaire — `/employe/remuneration`

Carte « Reste à recevoir » = gagné − payé, trois mini-cartes (gagné = cumul du dernier enregistrement de rémunération, payé, heures totales), **historique des rémunérations** (date, durée, salaire du jour, cumul) et **historique des paiements reçus** (date · note, +montant).

N. Matériel — `/employe/materiel`

Onglets « Liste » (recherche par nom/catégorie ; matériel disponible → bouton « Demander » avec note facultative) et « Mes emprunts » (demandes en attente → *Annuler* ; prêts en cours → *Signaler le retour* avec note ; historique). Mises à jour en temps réel.

O. Profil — `/employe/profil`

Lecture seule : identité, poste, département(s), email, téléphone, date d'embauche, salaire de base ; bascule thème clair/sombre. Le salarié **ne peut pas** modifier son profil ni son mot de passe depuis l'application.

4.5 Messagerie — composants partagés (`ChatPage`, `ChatConversation`)

- **Types de conversations** : *groupe* (créé par l'admin, que les salariés peuvent rejoindre/quitter librement ; option **« annonce »** = seul l'admin écrit) et *message direct* admin ↔ un salarié (identifiant déterministe `dm_{uidSalarié}` : une seule conversation par salarié).
- Admin : voit toutes les conversations, crée un groupe, ouvre un message direct (le salarié doit avoir un compte de connexion), peut **vider** une conversation, la **supprimer**, supprimer un message (« pour tout le monde »). Salarié : voit ses conversations + la liste des groupes à rejoindre.
- Messages : texte ≤ 2000 caractères, **réponse citée** (extrait ≤ 140 car.), **réactions emoji** (👍 ❤️ 😂 😮 ☹️ 🙏, bascule par utilisateur), copie du texte, 300 derniers messages chargés. Entrée envoie (appareils avec souris), Maj+Entrée = retour ligne.
- **« Vu »** : un accusé de lecture par personne et par conversation (`chats/{id}/reads/{uid}`), affiché sur *mon dernier message* pour ceux qui l'ont vu et n'ont pas répondu depuis.
- **Non-lus et badges** : l'heure de dernière lecture par conversation est stockée dans le `localStorage` du navigateur (`chat_read_{uid}`) ; le nombre exact de non-lus est obtenu par une requête de comptage. Aperçu du dernier message dans la liste (120 caractères).
- **Notifications locales Android** quand l'application est en arrière-plan (voir §10).

5. Parcours utilisateurs et processus métier

5.1 Mise en route de l'entreprise (admin)

1. Création du premier compte admin (aujourd'hui : manuellement dans la console Firebase + document `users/{uid} {role:"admin"}`, ou via l'écran de connexion avec le code d'invitation).
2. Paramètres → régler le salaire pour 8 h, la durée normale, l'heure d'entrée normale et la tolérance.
3. Lieux de travail → créer chaque département (position sur carte, rayon, géofencing).
4. Salariés → créer chaque salarié (identifiant, email de connexion, mot de passe/PIN, salaire de base, départements) et communiquer identifiant + mot de passe.

5.2 Journée d'un salarié (pointage)

1. Connexion avec son identifiant (`EMP-001`) + mot de passe.
2. Accueil → « POINTAGE ENTRÉE » → choix du département parmi les siens → GPS → contrôle du rayon → entrée enregistrée (heure serveur).
3. Pendant la journée : suivi de la durée et du salaire estimé en direct ; possibilité de consulter messages, matériel, etc.
4. « POINTAGE SORTIE » → GPS → contrôle du rayon par rapport au **même** département → calcul de la durée et du salaire de la session → cumul du solde.
5. Possibilité d'un « nouveau pointage » le même jour (pause déjeuner, etc.) ; le plafond journalier s'applique au **total du jour**.

5.3 Pointage à distance par l'admin

Si un salarié ne peut pas pointer (téléphone, GPS…), l'admin démarre le pointage (Présences → « Démarrer un pointage ») puis l'arrête (« Arrêter »). Aucune vérification de position ; la session est marquée « Admin » (entrée et/ou sortie) et l'identité de l'admin est conservée (`entryBy` / `exitBy`). Mêmes règles de salaire.

5.4 Rémunération et paiement

1. Chaque sortie crée un enregistrement de rémunération avec le cumul courant (`cumulativeSalary`).
2. **Gagné** = cumul du dernier enregistrement ; **Payé** = Σ des paiements ; **Reste à payer** = gagné − payé.
3. L'admin remet de l'argent en main propre puis l'enregistre (Rémunérations → « Payer »), sans pouvoir dépasser le reste à payer. Le salarié voit le paiement dans « Salaire ».
4. Le rapport mensuel (Rapports) agrège jours, heures et salaires sur la période et s'exporte en CSV.

5.5 Cycle de vie d'un prêt de matériel



- Le salarié ne peut demander que du matériel `DISPONIBLE` et pas deux fois la même demande en attente.
- **Approbation** (transaction) : refusée si la demande n'est plus `DEMANDE`, si le matériel n'existe plus, est déjà emprunté (« Ce matériel est déjà emprunté par quelqu'un d'autre. ») ou hors service ; sinon prêt → `EN_COURS` (date de prêt, décideur) et matériel → `EMPRUNTE` (prêt courant, emprunteur).
- **Confirmation du retour** (transaction) : possible depuis `EN_COURS` ou `RETOUR_DEMANDE` ; prêt → `RETOURNE` (date de retour, admin) ; matériel redevient `DISPONIBLE` si c'est bien ce prêt qui le détenait.

- Un matériel emprunté ne peut ni être mis hors service ni supprimé. Le nom du matériel et du salarié sont **copiés** dans le prêt : l'historique reste lisible même après suppression.

5.6 Messagerie

L'admin crée des groupes (éventuellement « annonce ») ; les salariés les rejoignent. Il ouvre des conversations privées avec un salarié. Chacun écrit, répond, réagit ; les accusés « Vu » et les compteurs de non-lus apparaissent en temps réel ; une notification locale est émise sur Android quand l'application est en arrière-plan.

5.7 Désactivation / suppression d'un salarié

- **Désactiver** (icône « power ») : `active = false` ; le salarié ne peut plus utiliser l'application (renvoyé vers `/login`) mais ses données sont conservées ; réactivable.
- **Supprimer** : supprime fiche + accès ; irréversible ; historique de pointage/paie conservé tel quel (orphelin).

6. Règles métier à conserver (calculs et contrôles)

6.1 Calcul du salaire d'une journée (`utils/salaryCalculator.ts`)

Paramètres : `dailySalary` (Ar pour une journée complète), `normalWorkMinutes` (défaut 480 = 8 h). Défauts : 20 000 Ar / 480 min, soit 2 500 Ar/h.

```
calculateDailySalary(minutes):
  si minutes ≤ 0                    → 0
  si minutes ≥ normalWorkMinutes    → dailySalary            (plafond : jamais plus)
  sinon                             → ROUND( minutes × dailySalary / normalWorkMinutes )   (arrondi
à l'Ar)
```

- **Salaire individuel** : `dailySalary` vaut `employees.baseDailySalary` du salarié concerné ; seule la durée normale (`normalWorkMinutes`) est globale. Le `settings.dailySalary` global ne sert que de valeur proposée à la création et de repli.
- **Durée d'une session** : `workedMinutes = max(0, round((sortie − entrée) / 60 000 ms))`.
- **Plusieurs sessions le même jour** : `salaire(session) = max(0, f(minutes déjà travaillées ce jour + durée) − f(minutes déjà travaillées ce jour))` où `f` = fonction ci-dessus et « minutes déjà travaillées » = Σ des sessions `TERMINEE` du même jour. Le plafond s'applique donc au total quotidien.
- **Jour de rattachement** : la date de l'**entrée** (une session à cheval sur minuit reste sur le jour d'entrée).
- **Cumul** : à chaque sortie, un enregistrement `salary_records` est créé avec `cumulativeSalary = solde précédent + salaire de la session` ; le solde précédent = `cumulativeSalary` du dernier enregistrement du salarié (tri par `createdAt` décroissant).

6.2 Contrôle GPS (géofencing)

- Distance de Haversine (rayon terrestre 6 371 000 m) entre la position du salarié et le centre du département choisi.
- Le contrôle est appliqué à l'**entrée** et à la **sortie**. Si le géofencing du département est **actif** et que la distance > `radiusMeters` → erreur « *GpsOutOfRangeError* » : **rien n'est enregistré**. Si le géofencing est **désactivé**, le pointage est accepté mais le contrôle (distance, `withinRadius`) est quand même enregistré.
- Le résultat est conservé dans le pointage (`entryGpsCheck` / `exitGpsCheck`) : position du salarié, position du département, rayon, distance arrondie, booléen dans/hors rayon.
- La position (`latitude, longitude, altitude, accuracy, address?`) est fournie **par le client** ; l'adresse vient d'un géocodage inverse OpenStreetMap (facultatif, délai 5 s, jamais bloquant). Le département de sortie = celui de l'entrée (`departmentId` du pointage).
- Défaut du rayon : 100 m. Un salarié ne peut pointer que dans les départements qui lui sont affectés (contrôle côté interface).

6.3 Autres règles

- **Une seule session ouverte** par salarié à la fois (« Vous avez déjà une entrée en cours. Pointez d'abord votre sortie. »). Plusieurs sessions terminées par jour sont permises.
- Sortie impossible sans entrée en cours (« Vous devez d'abord pointer votre entrée. » / « Aucune entrée en cours… »). Arrêt admin d'un pointage déjà terminé refusé.
- **Paiement** : montant > 0 ; ne peut pas dépasser le reste à payer (contrôle d'interface aujourd'hui ; à imposer côté serveur) ; date = jour courant ; note facultative (rognée).
- **Retard** : voir §4.3 A (première entrée du jour vs `normalEntryTime` + tolérance). Calculé à l'affichage, non stocké.
- **Salarié actif** requis pour accéder à l'application (vérifié à chaque résolution de session).

- **Département obligatoire** : un salarié doit avoir ≥ 1 département à la création/modification (message d'erreur sinon).
- **Mot de passe** : ≥ 6 caractères (contrainte Firebase Auth), identifiant salarié non modifiable après création.
- Messagerie : texte 1–2000 caractères ; un groupe « annonce » n'accepte que l'admin comme auteur.

7. Données actuelles dans Firebase

Firestore est **NoSQL, sans schéma ni contrainte** : les relations sont de simples identifiants texte, sans intégrité référentielle. Les horodatages sont des `Timestamp` Firestore. Les index composites ne sont pas déclarés (`firestore.indexes.json` vide).

7.1 Firebase Authentication

- Fournisseur **email + mot de passe**. Un compte par admin et par salarié. Pour un salarié, l'email est « technique » (champ `loginEmail`, ex. `emp001@votre-entreprise.local`) et le mot de passe est le code PIN choisi par l'admin.
- L'`uid` Firebase est la clé qui relie Auth, `users/{uid}`, les groupes de discussion (`memberUids`) et `createdBy/decidedBy/entryBy…`.

7.2 Collections et documents

Collection / document	ID du document	Champs (type) et remarques
`users`	`{uid}` Firebase	`role` ("admin"|"employee"), `employeeId` (string, salariés seulement), `inviteCode` (admins créés via le login).
`employees`	= `employeeId` (ex. "EMP-001")	`employeeId`, `firstName`, `lastName`, `address?`, `email` (pro), `phone?`, `position`, `departmentIds` (string[] → `workplaces`), `hireDate` (string), `loginEmail`, `authUid`, `active` (bool), `baseDailySalary` (number, Ar/jour), `createdAt`.
`workplaces`	auto	`name`, `latitude`, `longitude`, `radiusMeters`, `geofencingEnabled` (bool).
`attendance`	auto	`employeeId`, `date` ("YYYY-MM-DD"), `departmentId`, `departmentName` (copie), `entryTime`, `exitTime|null`, `workedMinutes`, `dailySalary`, `entryLocation|null` & `exitLocation|null` (`{latitude, longitude, altitude|null, accuracy, address?}`), `entryGpsCheck|null` & `exitGpsCheck|null` (`{employeeLatitude, employeeLongitude, departmentLatitude, departmentLongitude, radiusMeters, distance, withinRadius}`), `status` ("EN_COURS"|"TERMINEE"), `entrySource/exitSource` ("EMPLOYEE"|"ADMIN"), `entryBy/exitBy` (uid admin), `createdAt`, `updatedAt`.
`salary_records`	auto	`employeeId`, `attendanceId`, `date`, `workedMinutes`, `dailySalary`, `cumulativeSalary`, `createdAt`. Un enregistrement par pointage terminé.
`payments`	auto	`employeeId`, `amount`, `note`, `date` ("YYYY-MM-DD"), `createdBy` (uid admin), `createdAt`.
`settings` / `general`	"general" (unique)	`dailySalary` (20000), `normalWorkMinutes` (480), `toleranceMinutes` (10), `normalEntryTime` ("08:00"). Si absent : valeurs par défaut du code.
`equipment`	auto	`name`, `category?`, `serialNumber?`, `description?`, `status` ("DISPONIBLE"|"EMPRUNTE"|"HORS_SERVICE"), `currentLoanId?`, `currentBorrowerId?`, `currentBorrowerName?`, `createdAt`.

`equipmentLoans`	auto	`equipmentId`, `equipmentName` (copie), `employeeId`, `employeeName` (copie), `status` (6 valeurs, §5.5), `requestNote?`, `requestedAt`, `decidedAt?`, `decidedBy?`, `refusalReason?`, `loanedAt?`, `returnRequestedAt?`, `returnNote?`, `returnedAt?`, `returnConfirmedBy?`.
`chats`	auto, ou `dm_{uidSalarié}`	`type` ("group"|"direct"), `name`, `memberUids` (string[]), `createdBy`, `createdAt`, `announcement?` (bool), `directEmployeeUid?`, `directEmployeeName?`, aperçu : `lastMessageText` (≤120), `lastMessageAt`, `lastMessageBy` (uid), `lastMessageName`.
`chats/{id}/messages`	auto	`senderUid`, `senderName`, `senderRole`, `text` (≤2000), `createdAt`, `replyTo` (`{id, senderName, text≤140}`|null), `reactions` (`{emoji: uid[]}`).
`chats/{id}/reads`	`{uid}`	`uid`, `name`, `role`, `seenAt`.
`audit_logs`	auto	`userId`, `action`, `targetId`, `timestamp`. Actions réellement émises : `PAYMENT_CREATED`, `ATTENDANCE_CREATED`, `ATTENDANCE_UPDATED` (le type TypeScript déclare aussi `EMPLOYEE_CREATED/UPDATED/DISABLED`, `SETTINGS_UPDATED`, jamais émises). Append-only.
`admin_invite` / `main`	"main"	`code` (string) — créé à la main dans la console ; jamais lisible par le client ; sert uniquement aux règles de création d'admin.

7.3 Relations (logiques, non imposées)

```
users.employeeId ──────────► employees (id = employeeId)        employees.authUid ──► users (uid)
employees.departmentIds[] ─► workplaces.id                        (relation N–N stockée en
tableau)
attendance.employeeId ─────► employees          attendance.departmentId ─► workplaces
salary_records.attendanceId ► attendance        salary_records.employeeId ► employees
payments.employeeId ───────► employees          equipmentLoans.equipmentId ► equipment
equipmentLoans.employeeId ─► employees          equipment.currentLoanId ──► equipmentLoans
chats.memberUids[] ────────► users.uid          messages.senderUid ────────► users.uid
```

8. Règles de sécurité et contrôles d'accès actuels

Fichier `firestore.rules`. Fonctions utilitaires : `isSignedIn()`, `isAdmin()` (le document `users/{auth.uid}.role == 'admin'`), `isEmployee()`, `myEmployeeId()`.

Chemin	Règle actuelle
`users/{uid}`	Lecture : soi-même ou admin. Écriture : admin. **Création** aussi autorisée si `request.auth.uid == uid`, champs exactement `role` + `inviteCode`, `role == 'admin'` et `inviteCode == admin_invite/main.code`.
`admin_invite/*`	Aucun accès client (`allow read, write: if false`) ; lu seulement par les règles.
`employees/{employeeId}`	Lecture : admin, ou salarié pour **sa** fiche. Création/modification/suppression : admin.
`attendance/{id}`	Lecture : admin ou propriétaire. Création : admin, ou salarié pour lui-même avec `status == 'EN_COURS'` et `exitTime == null`. Mise à jour : admin, ou salarié propriétaire **uniquement** pour passer `EN_COURS → TERMINEE` sans changer `employeeId`, `entryTime`, `entryLocation`. Suppression : admin.
`salary_records/{id}`	Lecture : admin ou propriétaire. Création : admin, ou salarié pour lui-même (⚠ montant calculé par le client). Modification/suppression : admin.
`payments/{id}`	Lecture : admin ou propriétaire. Écriture : admin seulement.
`workplaces/*`, `settings/*`, `equipment/*`	Lecture : tout utilisateur connecté. Écriture : admin.
`equipmentLoans/{id}`	Lecture : admin ou propriétaire. Création : admin, ou salarié pour lui-même avec statut `DEMANDE`. Mise à jour : admin, ou salarié propriétaire pour (a) `DEMANDE → ANNULEE` (champs `status, decidedAt` seuls) ou (b) `EN_COURS → RETOUR_DEMANDE` (champs `status, returnRequestedAt, returnNote` seuls). Suppression : admin.
`chats/{id}`	Lecture : admin, ou connecté si groupe ou membre. Création/suppression : admin. Mise à jour : admin, ou connecté pour : *rejoindre* un groupe (ajoute seulement son uid, +1 membre), *quitter* (retire seulement son uid, −1), ou mise à jour de l'aperçu du dernier message par un membre (champs `lastMessage*` uniquement).
`chats/{id}/messages/{m}`	Lecture : admin ou membre. Création : connecté, admin ou membre (si le chat n'est pas « annonce »), avec `senderUid == auth.uid` et texte string de 1 à 2000. Mise à jour : admin ou membre, **seulement** le champ `reactions`. Suppression : admin.
`chats/{id}/reads/{uid}`	Lecture : admin ou membre. Création/maj : connecté (admin ou membre) uniquement sur **son** document. Suppression : admin.
`audit_logs/{id}`	Lecture : admin. Création : tout connecté. Modification/suppression : interdites.

> **À retenir pour la migration :** toutes ces règles doivent être **réimplémentées côté serveur** (middleware d'autorisation + contrôles par endpoint). Le backend doit même être **plus strict** que les règles actuelles sur deux points : (1) le **salaire** et le **cumul** sont calculés par le serveur, jamais fournis par le client ; (2) le **contrôle GPS** et la **durée** sont vérifiés/calculés par le serveur avec son horloge.

9. Flux de données UI ⇄ Firebase (opérations par page)

Toutes les opérations passent par `src/services/*`. Légende : **R** = lecture, **C** = création, **U** = modification, **D** = suppression, **RT** = abonnement temps réel (`onSnapshot`), **TX** = transaction/batch.

Page	Fonction de service	Opération Firestore / Auth
Login	`loginWithIdentifier`	R `employees/{id}` (pour `loginEmail`) puis Auth `signInWithEmailAndPassword`.
	`createAdminAccount`	Auth `createUserWithEmailAndPassword` (instance secondaire) + C `users/{uid}` ; rollback du compte Auth si l'écriture échoue.
	AuthContext → `getAppUser`	R `users/{uid}`, puis pour un salarié R `employees/{id}` et refus si `active === false` ou fiche absente. Écoute `onAuthStateChanged`.
Salariés (admin)	`listEmployees`	R `employees` trié par `lastName`.
	`createEmployee`	Auth create (secondaire) + C `users/{uid}` + C `employees/{employeeId}` (`createdAt` serveur).
	`updateEmployee`, `setEmployeeActive`	U `employees/{id}`.
	`deleteEmployee`	R `employees/{id}` (pour `authUid`), D `employees/{id}`, D `users/{uid}`.
	`changeEmployeePassword` / `sendEmployeePasswordReset`	Auth `signInWithEmailAndPassword` + `updatePassword` (instance secondaire) / `sendPasswordResetEmail`.
Pointage salarié (Accueil)	`listEmployeeSessions`	R `attendance` où `employeeId == X` et `date == jour` (tri en mémoire).
	`clockIn`	R sessions du jour (refus si EN_COURS) ; contrôle GPS en mémoire ; C `attendance`.
	`clockOut` → `finalizeAttendance`	R sessions du jour ; **TX** : R dernier `salary_records` (solde), U `attendance` (sortie, durée, salaire, `TERMINEE`), C `salary_records` (cumul).
	`getCumulativeBalance`	R dernier `salary_records` du salarié (`orderBy createdAt desc limit 1`).
	`getEmployee`, `listWorkplaces`, `getSettings`	R `employees/{id}`, R `workplaces`, R `settings/general`.
	`listAttendanceHistory` (Historique)	R `attendance` du salarié, `orderBy date desc limit 60`.
Présences / Carte (admin)	`listAttendanceForDate`	R `attendance` où `date == D`.
	`adminClockIn`	R sessions du jour ; C `attendance` ; C `audit_logs`.
	`adminClockOut`	R `attendance/{id}` ; mêmes TX que `clockOut` ; C `audit_logs`.
Rémunérations, Dashboard, Salaire	`createPayment`	C `payments` + C `audit_logs` (`PAYMENT_CREATED`).
	`listPayments`	R `payments` du salarié, `orderBy createdAt desc`.
	`listSalaryHistory` (page Salaire)	R `salary_records` du salarié, `orderBy date desc`.
Rapports	`listSalaryForRange`	R `salary_records` avec `date >= début` et `<= fin`, `orderBy date asc` ; agrégation faite dans le navigateur.

Lieux de travail	`listWorkplaces / createWorkplace / updateWorkplace / deleteWorkplace`	R / C / U / D `workplaces`.
Paramètres	`getSettings / updateSettings`, `verifyAdminPassword`, `resetAllData`	R / U (merge) `settings/general` ; Auth `reauthenticateWithCredential` ; purge par lots de 400 de `chats` (+ sous-collections), `attendance, salary_records, payments, equipmentLoans, equipment, employees` et `users` sauf admins.
Matériel	`subscribeEquipment / subscribeLoans / subscribePendingCount`	**RT** sur `equipment`, `equipmentLoans` (tous ou filtrés par salarié), et comptage des prêts `status in [DEMANDE, RETOUR_DEMANDE]`.
	`create/update/delete Equipment`, `setOutOfService`	C / U / D `equipment` ; refus si EMPRUNTE.
	`requestLoan, cancelRequest, reportReturn, approveLoan, refuseLoan, confirmReturn`	C/U `equipmentLoans` ; `approveLoan` et `confirmReturn` en **TX** (prêt + matériel).
Discussions	`subscribeChats`	**RT** `chats` : admin = tous ; salarié = (`memberUids array-contains uid`) ∪ (`type == group`).
	`subscribeMessages / subscribeReads`	**RT** `messages` (`orderBy createdAt`, 300 derniers) et `reads`.
	`createGroup, openDirectChat, joinGroup, leaveGroup, sendMessage, toggleReaction, markSeen`	C/U `chats` ; `sendMessage` = **batch** (C message + U aperçu du chat) ; `joinGroup/leaveGroup` = `arrayUnion/arrayRemove` sur `memberUids` ; `toggleReaction` = arrayUnion/Remove sur `reactions.{emoji}` ; `markSeen` = set `reads/{uid}`.
	`clearConversation, deleteChat, deleteMessage` ; ChatContext	D par paquets (messages, reads) ; comptage des non-lus par `getCountFromServer` (`createdAt > dernière lecture`).

10. Notifications, recherches, filtres, validations

10.1 Notifications

- **Notifications locales Android uniquement** (`services/notify.ts`, plugin Capacitor Local Notifications) : canal « messages » (importance maximale, vibration). Déclenchées côté **client** quand un nouveau message d'un *autre* utilisateur arrive alors que la page est masquée (`document.hidden`) ; id stable par conversation (la nouvelle remplace l'ancienne) ; un clic ouvre `/admin/discussions` ou `/employe/discussions`. Titre : nom du groupe, « Administrateur » ou nom du salarié ; corps : « Nom : texte » pour un groupe.
- **Limite connue** : application fermée (tuée) = **aucune notification** (pas de serveur de push, pas de FCM). Le nouveau backend ne change pas ce comportement sauf demande d'évolution (push FCM/Web Push hors périmètre).
- **Badges** : non-lus Discussions (admin et salarié), demandes de matériel à traiter (admin). **Toasts** (5 s) pour succès/erreurs ; fenêtre de confirmation pour toute action destructrice.

10.2 Recherches et filtres existants

- Présences, Carte : date ; Carte : salarié, type de marqueur, affichage des lieux.
- Rapports : période (début/fin).
- Matériel admin : filtres Historique par matériel et par statut. Matériel salarié : recherche texte (nom + catégorie, insensible à la casse).
- Lieux de travail : recherche d'adresse (géocodage Nominatim, 5 résultats) et clic sur carte.
- Tri : salariés par nom ; rapports par salaire décroissant ; chats par dernier message ; prêts par date de demande décroissante.
- Aucune pagination (limites : 60 pointages en historique, 300 messages affichés ; tout le reste est chargé en entier).

10.3 Validations côté interface (à reproduire et à imposer côté serveur)

Formulaire	Règles
Connexion admin (création)	mot de passe ≥ 6, confirmation identique, code d'invitation requis.
Salarié	identifiant, prénom, nom, email de connexion obligatoires ; salaire ≥ 0 (nombre) ; ≥ 1 département ; mot de passe ≥ 6 à la création ; changement de mot de passe : nouveau ≥ 6.
Lieu de travail	nom requis ; latitude ∈ [−90, 90], longitude ∈ [−180, 180] ; rayon > 0.
Paiement	montant > 0 et ≤ reste à payer.
Paramètres	nombres (salaire, heures, tolérance), heure `HH:mm`.
Matériel	nom requis ; catégorie, n° de série, description facultatifs.
Messagerie	nom de groupe requis ; message non vide, ≤ 2000 caractères.

11. Dépendances et services externes

Service / paquet	Usage actuel	Devenir
Firebase Auth	Authentification email/mot de passe, gestion des comptes.	**Remplacer** (backend).
Cloud Firestore	Base de données + temps réel + règles.	**Remplacer** (PostgreSQL + API).
Firebase Analytics	Initialisé (`getAnalytics`) sans événement personnalisé visible.	**Supprimer** (ou autre outil si besoin).
Vercel	Hébergement du frontend (SPA) ; l'app Android charge cette URL.	Conserver ou migrer ; prévoir l'URL du backend (CORS).
OpenStreetMap — tuiles et **Nominatim**	Fond de carte « Plan » ; géocodage direct (recherche d'adresse) et inverse (adresse du pointage). Pas de clé API.	Conserver (côté client) ; respecter la politique d'usage de Nominatim (≈ 1 requête/s, identification) ou prévoir un proxy/instance dédiée.
Esri ArcGIS (tuiles)	Fonds « Satellite » et « Hybride ». Pas de clé API.	Conserver.
Capacitor (Android)	Géolocalisation native (Google Play Services, avec repli `LocationManager`, délai 20 s), notifications locales, WebView sur l'URL Vercel.	Conserver.
Paquets npm	`react, react-dom, react-router-dom, leaflet, react-leaflet, lucide-react, recharts, firebase, @capacitor/*` ; dev : `vite, typescript, tailwindcss, postcss, autoprefixer, @vitejs/plugin-react, types`.	Retirer `firebase` ; `recharts` semble inutilisé (à vérifier avant retrait).

Aucun service d'email transactionnel, de stockage de fichiers, de paiement ou de SMS n'est utilisé. L'envoi d'un email de réinitialisation de mot de passe repose aujourd'hui sur Firebase (`sendPasswordResetEmail`) et disparaîtra avec lui.

12. Points d'attention et anomalies constatées dans le code actuel

Constats issus de la lecture du code. Ils ne sont pas tous à « reproduire » : pour chacun, le développeur doit trancher avec le responsable du projet avant de coder.

#	Constat	Recommandation pour le nouveau backend
1	**Cumul du solde non atomique.** Dans `finalizeAttendance`, la lecture du solde précédent (`getCumulativeBalance`) se fait par une requête *hors* transaction ; deux sorties simultanées du même salarié pourraient donner un cumul faux.	Transaction SQL avec verrou (`SELECT … FOR UPDATE` sur la fiche salarié) ; ou ne plus stocker le cumul (somme à la demande).
2	**Salaire, durée et GPS calculés/fournis par le client** ; les règles autorisent le salarié à créer lui-même un `salary_records` avec n'importe quel montant. Position GPS falsifiable (fausse localisation).	Tout calculer côté serveur avec l'horloge serveur ; interdire au client d'envoyer durée/salaire ; vérifier Haversine côté serveur ; stocker la précision (`accuracy`) et éventuellement la rejeter au-delà d'un seuil (à définir).
3	**Page Salaire du salarié** : « gagné » = `records[0].cumulativeSalary` avec tri par `date desc` ; si plusieurs sessions le même jour, l'ordre est indéterminé et le cumul affiché peut être celui d'une session intermédiaire. (Les pages admin trient par `createdAt`.)	Un seul calcul serveur du « gagné » (somme ou dernier cumul par date de création) exposé par l'API.
4	**Modifier un salarié le réactive** : `handleSubmit` appelle `updateEmployee(id, {…, active: true})` ; éditer un salarié désactivé le remet actif.	Ne pas modifier `active` dans le PATCH d'édition (il a son endpoint dédié).
5	**Email de connexion modifié** en édition : seul le document `employees` est mis à jour, pas le compte Auth ; le login par identifiant utiliserait alors un email inexistant dans Auth.	Décider si l'identifiant de connexion d'un salarié est son code (recommandé : plus d'« email Firebase » technique) ; si oui supprimer ce champ de l'UI.
6	**Résolution du login par identifiant** : `resolveEmployeeEmail` lit `employees/{id}` *avant* authentification, ce que les règles du dépôt interdisent (lecture réservée à l'admin/propriétaire). À vérifier : les règles déployées en production peuvent différer de celles du dépôt.	Le nouvel endpoint de login résout lui-même l'identifiant côté serveur ; message d'erreur unique.
7	**Suppressions sans cascade** : supprimer un salarié laisse pointages/rémunérations/paiements/prêts orphelins et le compte Auth ; supprimer un lieu laisse l'ID dans `departmentIds` des salariés ; supprimer un matériel garde les prêts (volontaire, noms copiés).	Clés étrangères explicites (§14). Pour un salarié : préférer la désactivation, ou suppression avec conservation de l'historique (`ON DELETE RESTRICT` + « archivage »). À décider.
8	**Session « en cours » oubliée** : aucune clôture automatique ; le bouton « Arrêter » admin n'apparaît que pour la date du jour (`isToday`) : une session ouverte la veille ne peut être ni arrêtée par l'UI ni remplacée (le salarié est bloqué par « entrée déjà en cours »). Aucune UI pour corriger ou supprimer un pointage/paiement, bien que `deletePayment` existe en service.	Prévoir (si validé) : clôture automatique configurable ou action admin « corriger/clôturer » sur toute date, avec audit.
9	**Non-lus de la messagerie stockés dans `localStorage`** (par navigateur), donc non synchronisés entre appareils.	Stocker l'état de lecture côté serveur (table `chat_reads`).

10	**Audit partiel** : seuls 3 types d'actions sont journalisés ; `audit_logs` n'est consultable dans aucune page ; tout utilisateur connecté peut y écrire.	Journaliser côté serveur toutes les actions sensibles ; page de consultation facultative.
11	**N+1 requêtes** (Dashboard et Rémunérations : 2 requêtes par salarié) ; agrégations (rapports, dashboard) faites dans le navigateur.	Endpoints agrégés SQL (§16).
12	**Configuration en dur** dans `services/firebase.ts` et README partiellement obsolète (il décrit la création manuelle des comptes salariés alors que `createEmployeeAccount` existe).	Variables d'environnement pour l'URL de l'API ; mettre le README à jour.
13	**« Réinitialiser les données »** supprime aussi les salariés mais pas leurs comptes Auth ; l'opération est faite par le client en plusieurs lots (non atomique).	Endpoint serveur transactionnel, protégé par re-saisie du mot de passe, avec journalisation.
14	Date d'embauche saisie en texte libre ; `hireDate` sans format imposé.	Type `date` en base ; migration tolérante (valeurs invalides → NULL).
15	Anciennes données de lieux possiblement avec coordonnées texte/absentes (le code les convertit en `NaN`).	Nettoyer/valider à la migration (rejeter ou corriger) avant d'imposer les contraintes `NOT NULL`.

13. Architecture cible : backend + PostgreSQL

13.1 Principes

- **Le backend devient l'unique point d'accès aux données.** Le frontend n'accède plus jamais à la base ; il appelle une **API REST JSON** (+ un canal temps réel pour la messagerie et le matériel).
- **Les règles métier migrent côté serveur** (calcul du salaire, contrôle GPS, transactions de prêt, vérification des soldes de paiement) ; le frontend ne conserve qu'un affichage et un *aperçu* (salaire estimé en direct).
- **Sécurité par défaut** : chaque endpoint vérifie l'authentification, le rôle et la propriété de la ressource (un salarié n'accède qu'à ses données).
- **Intégrité en base** : clés étrangères, contraintes `CHECK`, index uniques partiels, transactions.

13.2 Stack recommandée (modifiable)

Élément	Recommandation
Langage / framework	Node.js 20+ avec **TypeScript** (même langage que le frontend, types partageables) — NestJS ou Express/Fastify. Toute autre stack est acceptable si elle respecte ce document.
Base de données	**PostgreSQL 15+**. Fuseau de session `Indian/Antananarivo` pour les « jours » ; colonnes d'horodatage en `timestamptz` (UTC).
Accès aux données	ORM/Query builder avec **migrations versionnées** (Prisma, Drizzle, Knex ou TypeORM). Les contraintes du §14 doivent être dans les migrations SQL.
Validation	Schémas par endpoint (zod / class-validator / Joi) : types, bornes, longueurs.
Authentification	Mots de passe hachés **argon2id** (ou bcrypt coût ≥ 12) ; **JWT d'accès** court + **refresh token** rotatif (§15).
Temps réel	WebSocket (ou Server-Sent Events) + éventuellement `LISTEN/NOTIFY` PostgreSQL si plusieurs instances (§17).
Déploiement	HTTPS obligatoire (la WebView Android et la géolocalisation l'exigent), CORS limité aux origines du frontend, variables d'environnement pour tous les secrets, sauvegardes PostgreSQL quotidiennes (`pg_dump`/PITR), journaux applicatifs.

13.3 Structure suggérée du backend

```
backend/
  src/
    main.ts                    démarrage, CORS, helmet, rate limit, WebSocket
    config/                    lecture des variables d'environnement (DATABASE_URL, JWT_SECRET…)
    auth/                      login, refresh, logout, register-admin, guards (rôle, propriété)
    employees/                 CRUD salariés + comptes + mot de passe
    workplaces/                lieux de travail (départements)
    attendance/                clock-in / clock-out / admin (GPS + salaire en transaction)
    salary/ payments/          soldes, historique, paiements, rapports
    settings/                  paramètres de rémunération
    equipment/ loans/          matériel + cycle de prêt
    chat/                      conversations, messages, réactions, lectures + passerelle temps
réel
    dashboard/ admin/          agrégats, réinitialisation, audit
    common/                    erreurs, validation, horloge (jour de travail), Haversine, audit
  migrations/                  SQL versionné
  scripts/                     import Firestore → PostgreSQL, création du 1er admin
```

13.4 Variables d'environnement minimales

```
DATABASE_URL=postgres://user:pass@host:5432/pointage
JWT_ACCESS_SECRET=…        JWT_ACCESS_TTL=15m
REFRESH_TOKEN_TTL=30d      CORS_ORIGINS=https://suivie-
fortico.vercel.app,capacitor://localhost,http://localhost:5173
APP_TIMEZONE=Indian/Antananarivo
ADMIN_SIGNUP_CODE=…        (ou table admin_invites, §14)
GPS_MAX_ACCURACY_M=…       (optionnel, à décider)
```

14. Schéma PostgreSQL détaillé

Modélisation fidèle aux collections actuelles, avec **intégrité référentielle** ajoutée. Les montants sont des **entiers en Ariary** (le code actuel arrondit déjà à l'unité). Les noms sont en `snake_case` ; le backend peut renvoyer du `camelCase` pour limiter les changements du frontend (§19).

14.1 Vue des relations

```
users 1──0..1 employees        employees N──N workplaces  (via employee_workplaces)
employees 1──N attendance      attendance 1──0..1 salary_records
employees 1──N payments        employees 1──N equipment_loans     equipment 1──N equipment_loans
chats 1──N chat_messages       chats N──N users (chat_members)    chat_messages 1──N
message_reactions
chats 1──N chat_reads          users 1──N refresh_tokens          users 1──N audit_logs
```

14.2 Script de création (référence)

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;   -- gen_random_uuid()

CREATE TYPE user_role         AS ENUM ('admin','employee');
CREATE TYPE punch_source      AS ENUM ('EMPLOYEE','ADMIN');
CREATE TYPE attendance_status AS ENUM ('EN_COURS','TERMINEE');
CREATE TYPE equipment_status  AS ENUM ('DISPONIBLE','EMPRUNTE','HORS_SERVICE');
CREATE TYPE loan_status       AS ENUM
('DEMANDE','REFUSEE','ANNULEE','EN_COURS','RETOUR_DEMANDE','RETOURNE');
CREATE TYPE chat_type         AS ENUM ('group','direct');

-- Paramètres (une seule ligne)
CREATE TABLE settings (
  id                   smallint PRIMARY KEY DEFAULT 1 CHECK (id = 1),
  daily_salary         integer NOT NULL DEFAULT 20000 CHECK (daily_salary >= 0),
  normal_work_minutes  integer NOT NULL DEFAULT 480   CHECK (normal_work_minutes > 0),
  tolerance_minutes    integer NOT NULL DEFAULT 10    CHECK (tolerance_minutes >= 0),
  normal_entry_time    time    NOT NULL DEFAULT '08:00',
  updated_at           timestamptz NOT NULL DEFAULT now()
);
INSERT INTO settings DEFAULT VALUES;

CREATE TABLE workplaces (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name                text NOT NULL CHECK (length(btrim(name)) > 0),
  latitude            double precision NOT NULL CHECK (latitude  BETWEEN -90  AND 90),
  longitude           double precision NOT NULL CHECK (longitude BETWEEN -180 AND 180),
  radius_meters       integer NOT NULL DEFAULT 100 CHECK (radius_meters > 0),
  geofencing_enabled  boolean NOT NULL DEFAULT true,
  created_at          timestamptz NOT NULL DEFAULT now()
);

-- Clé métier = code salarié (immuable après création, comme l'ID de document actuel)
CREATE TABLE employees (
  employee_code     varchar(30) PRIMARY KEY,                -- ex. 'EMP-001'
  first_name        text NOT NULL,
  last_name         text NOT NULL,
  address           text,
  email             text,                                   -- email professionnel (facultatif)
  phone             text,
  position          text NOT NULL DEFAULT '',
  hire_date         date,
  active            boolean NOT NULL DEFAULT true,
```

```
  base_daily_salary integer NOT NULL DEFAULT 0 CHECK (base_daily_salary >= 0),
  created_at        timestamptz NOT NULL DEFAULT now(),
  updated_at        timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE users (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email          text,                                        -- obligatoire pour admin, facultatif
pour salarié
  password_hash  text NOT NULL,
  role           user_role NOT NULL,
  employee_code  varchar(30) UNIQUE REFERENCES employees(employee_code) ON DELETE CASCADE,
  created_at     timestamptz NOT NULL DEFAULT now(),
  last_login_at  timestamptz,
  CHECK ((role = 'employee') = (employee_code IS NOT NULL)),
  CHECK (role = 'employee' OR email IS NOT NULL)
);
CREATE UNIQUE INDEX uq_users_email ON users (lower(email)) WHERE email IS NOT NULL;

CREATE TABLE employee_workplaces (
  employee_code  varchar(30) NOT NULL REFERENCES employees(employee_code) ON DELETE CASCADE,
  workplace_id   uuid        NOT NULL REFERENCES workplaces(id)           ON DELETE CASCADE,
  PRIMARY KEY (employee_code, workplace_id)
);

CREATE TABLE attendance (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  employee_code    varchar(30) NOT NULL REFERENCES employees(employee_code) ON DELETE RESTRICT,
  work_date        date NOT NULL,                             -- jour de l'ENTRÉE (fuseau
Indian/Antananarivo)
  workplace_id     uuid REFERENCES workplaces(id) ON DELETE SET NULL,
  department_name  text NOT NULL,                             -- copie du nom au moment du pointage
  entry_time       timestamptz NOT NULL,
  exit_time        timestamptz,
  worked_minutes   integer NOT NULL DEFAULT 0 CHECK (worked_minutes >= 0),
  daily_salary     integer NOT NULL DEFAULT 0 CHECK (daily_salary >= 0),   -- salaire de CETTE
session
  status           attendance_status NOT NULL DEFAULT 'EN_COURS',
  entry_source     punch_source NOT NULL DEFAULT 'EMPLOYEE',
  exit_source      punch_source,
  entry_by         uuid REFERENCES users(id) ON DELETE SET NULL,   -- admin (pointage à distance)
  exit_by          uuid REFERENCES users(id) ON DELETE SET NULL,
  entry_lat double precision, entry_lng double precision, entry_altitude double precision,
  entry_accuracy double precision, entry_address text,
  exit_lat  double precision, exit_lng  double precision, exit_altitude  double precision,
  exit_accuracy  double precision, exit_address  text,
  entry_gps_check  jsonb,   -- {employeeLatitude, employeeLongitude, departmentLatitude,
departmentLongitude, radiusMeters, distance, withinRadius}
  exit_gps_check   jsonb,
  created_at       timestamptz NOT NULL DEFAULT now(),
  updated_at       timestamptz NOT NULL DEFAULT now(),
  CHECK (status = 'EN_COURS' OR exit_time IS NOT NULL),
  CHECK (exit_time IS NULL OR exit_time >= entry_time)
);
-- Jamais deux sessions ouvertes pour le même salarié
CREATE UNIQUE INDEX uq_attendance_one_open ON attendance (employee_code) WHERE status =
'EN_COURS';
CREATE INDEX ix_attendance_date          ON attendance (work_date);
CREATE INDEX ix_attendance_emp_date      ON attendance (employee_code, work_date DESC);

CREATE TABLE salary_records (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  employee_code      varchar(30) NOT NULL REFERENCES employees(employee_code) ON DELETE RESTRICT,
  attendance_id      uuid NOT NULL UNIQUE REFERENCES attendance(id) ON DELETE RESTRICT,
  work_date          date NOT NULL,
  worked_minutes     integer NOT NULL CHECK (worked_minutes >= 0),
  daily_salary       integer NOT NULL CHECK (daily_salary >= 0),
  cumulative_salary  integer NOT NULL CHECK (cumulative_salary >= 0),
  created_at         timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ix_salary_emp_created ON salary_records (employee_code, created_at DESC);
CREATE INDEX ix_salary_date        ON salary_records (work_date);

CREATE TABLE payments (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  employee_code  varchar(30) NOT NULL REFERENCES employees(employee_code) ON DELETE RESTRICT,
  amount         integer NOT NULL CHECK (amount > 0),
  note           text NOT NULL DEFAULT '',
  payment_date   date NOT NULL,
  created_by     uuid REFERENCES users(id) ON DELETE SET NULL,
  created_at     timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ix_payments_emp ON payments (employee_code, created_at DESC);

CREATE TABLE equipment (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name           text NOT NULL CHECK (length(btrim(name)) > 0),
  category       text, serial_number text, description text,
  status         equipment_status NOT NULL DEFAULT 'DISPONIBLE',
  created_at     timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE equipment_loans (
  id                    uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  equipment_id          uuid REFERENCES equipment(id) ON DELETE SET NULL,   -- l'historique survit
à la suppression
  equipment_name        text NOT NULL,                                       -- copie
  employee_code         varchar(30) REFERENCES employees(employee_code) ON DELETE SET NULL,
  employee_name         text NOT NULL,                                       -- copie
  status                loan_status NOT NULL DEFAULT 'DEMANDE',
  request_note          text,
  requested_at          timestamptz NOT NULL DEFAULT now(),
  decided_at            timestamptz,  decided_by uuid REFERENCES users(id) ON DELETE SET NULL,
  refusal_reason        text,
  loaned_at             timestamptz,
  return_requested_at   timestamptz,  return_note text,
  returned_at           timestamptz,  return_confirmed_by uuid REFERENCES users(id) ON DELETE SET
NULL
);
-- Un matériel ne peut avoir qu'un seul prêt actif à la fois
CREATE UNIQUE INDEX uq_loan_one_active ON equipment_loans (equipment_id) WHERE status IN
('EN_COURS','RETOUR_DEMANDE');
CREATE INDEX ix_loans_status ON equipment_loans (status);
CREATE INDEX ix_loans_emp    ON equipment_loans (employee_code, requested_at DESC);

CREATE TABLE chats (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  type                chat_type NOT NULL,
  name                text NOT NULL DEFAULT '',
  announcement        boolean NOT NULL DEFAULT false,
  created_by          uuid REFERENCES users(id) ON DELETE SET NULL,
  direct_user_id      uuid UNIQUE REFERENCES users(id) ON DELETE CASCADE,   -- salarié du message
direct (1 conversation / salarié)
  created_at          timestamptz NOT NULL DEFAULT now(),
```

```
  last_message_text   text, last_message_at timestamptz,
  last_message_by     uuid REFERENCES users(id) ON DELETE SET NULL, last_message_name text,
  CHECK ((type = 'direct') = (direct_user_id IS NOT NULL)),
  CHECK (type = 'group' OR announcement = false)
);

CREATE TABLE chat_members (
  chat_id    uuid NOT NULL REFERENCES chats(id) ON DELETE CASCADE,
  user_id    uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  joined_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (chat_id, user_id)
);

CREATE TABLE chat_messages (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  chat_id          uuid NOT NULL REFERENCES chats(id) ON DELETE CASCADE,
  sender_user_id   uuid REFERENCES users(id) ON DELETE SET NULL,
  sender_name      text NOT NULL,
  sender_role      user_role NOT NULL,
  body             text NOT NULL CHECK (char_length(body) BETWEEN 1 AND 2000),
  reply_to_id      uuid REFERENCES chat_messages(id) ON DELETE SET NULL,
  reply_snapshot   jsonb,                       -- {id, senderName, text (≤140)} conservé même si
le message cité est supprimé
  created_at       timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX ix_msg_chat_created ON chat_messages (chat_id, created_at);

CREATE TABLE message_reactions (
  message_id  uuid NOT NULL REFERENCES chat_messages(id) ON DELETE CASCADE,
  user_id     uuid NOT NULL REFERENCES users(id)         ON DELETE CASCADE,
  emoji       text NOT NULL,
  PRIMARY KEY (message_id, user_id, emoji)
);

CREATE TABLE chat_reads (               -- accusé « Vu » + base du compteur de non-lus
  chat_id   uuid NOT NULL REFERENCES chats(id) ON DELETE CASCADE,
  user_id   uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name      text NOT NULL, role user_role NOT NULL,
  seen_at   timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (chat_id, user_id)
);

CREATE TABLE refresh_tokens (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  text NOT NULL UNIQUE,
  expires_at  timestamptz NOT NULL,
  revoked_at  timestamptz,
  created_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE admin_invites (            -- remplace admin_invite/main ; ou simple variable
d'environnement
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  code_hash   text NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  revoked_at  timestamptz
);

CREATE TABLE audit_logs (               -- append-only : pas d'UPDATE/DELETE pour le rôle
applicatif
  id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
  user_id     uuid REFERENCES users(id) ON DELETE SET NULL,
  action      text NOT NULL,
  target_id   text,
  details     jsonb,
  created_at  timestamptz NOT NULL DEFAULT now()
);
REVOKE UPDATE, DELETE ON audit_logs FROM app_user;   -- rôle PostgreSQL de l'application
```

14.3 Règles d'intégrité à retenir

Contrainte	Mécanisme
Un salarié a au plus une session `EN_COURS`	Index unique partiel `uq_attendance_one_open` (+ gestion de l'erreur `23505` → `409 ALREADY_CLOCKED_IN`).
Un matériel n'a qu'un prêt actif	Index unique partiel `uq_loan_one_active` + transaction avec `SELECT … FOR UPDATE` sur le matériel.
Un pointage terminé a une heure de sortie ≥ entrée	`CHECK` sur `attendance`.
Un seul enregistrement de rémunération par pointage	`UNIQUE(attendance_id)`.
Rôle ↔ fiche salarié cohérents	`CHECK ((role='employee') = (employee_code IS NOT NULL))`.
Un message direct par salarié	`UNIQUE(direct_user_id)` (remplace l'identifiant `dm_{uid}`).
Historique financier non supprimable par cascade	`ON DELETE RESTRICT` sur `attendance`, `salary_records`, `payments` → supprimer un salarié ayant un historique est refusé : proposer « désactiver » (à valider, cf. §12 n°7).
Reste à payer jamais négatif	Vérification en transaction à chaque création de paiement (verrou sur la fiche salarié) : `Σpaiements + montant ≤ Σsalaires gagnés`.
Journal d'audit immuable	Droits SQL limités + aucun endpoint de modification.

> **Solde « gagné » :** deux options équivalentes. (a) Conserver `cumulative_salary` (fidèle à l'existant, nécessite un verrou lors de l'écriture) ; (b) ne plus s'appuyer dessus et utiliser `SUM(daily_salary)`. Dans les deux cas, **un seul calcul serveur** du « gagné » doit être exposé (cf. §12 n°3). La colonne doit au moins être remplie pour que l'historique affiché au salarié (« cumul » par ligne) reste identique.

15. Authentification et autorisation

15.1 Authentification

- **Identifiant de connexion** : un champ unique « Identifiant / Email » (comme aujourd'hui). Le serveur détermine le compte : si la valeur contient « @ » → recherche par email (admin, ou salarié ayant un email) ; sinon → code salarié (`EMP-001`, comparaison insensible à la casse recommandée). L'« email de connexion Firebase » des salariés n'a plus de raison d'être (cf. §12 n°5) : l'identifiant est le code salarié.
- **Mot de passe / PIN** : minimum 6 caractères (comportement actuel ; durcissement possible, à valider). Hachage **argon2id**. Message d'échec unique (« Identifiant ou mot de passe incorrect. ») pour ne pas révéler l'existence d'un compte ; limitation de débit (ex. 5 essais/15 min par IP + identifiant).
- **Salarié désactivé** (`employees.active = false`) ou fiche supprimée : connexion refusée et jetons existants invalidés (vérification à chaque requête ou à chaque rafraîchissement). Comportement actuel : renvoi vers `/login`.
- **Jetons** : JWT d'accès (≈ 15 min, contient `sub` = id utilisateur, `role`, `employeeCode`) en en-tête `Authorization: Bearer` ; **refresh token** aléatoire, stocké haché en base, rotatif, idéalement en cookie `HttpOnly; Secure; SameSite=Lax`. L'application Android charge le site web distant dans une WebView : le fonctionnement est identique à celui du navigateur.
- **Déconnexion** : révocation du refresh token. Persistance : la session doit survivre au rechargement de page (comme `onAuthStateChanged`).
- **Changement de mot de passe** d'un salarié par l'admin : l'API **ne demande plus l'ancien mot de passe** (levée de la limite du SDK client) ; tous les refresh tokens du salarié sont révoqués. L'envoi d'un email de réinitialisation n'est utile que si l'on stocke un email salarié et dispose d'un service d'envoi — **facultatif**, à décider.
- **Création d'un compte admin** depuis l'écran de connexion : conserver la fonctionnalité telle quelle (email, mot de passe, confirmation, **code d'invitation**) ; le serveur compare le code (stocké haché ou en variable d'environnement), applique une limitation de débit, journalise la création. Prévoir aussi un script d'installation `create-admin` pour le **tout premier admin**.
- **Réinitialisation des données** et autres actions destructrices : re-saisie du mot de passe de l'admin (`POST /admin/reset-data` avec `password`).

15.2 Autorisation (reprise de la matrice du §3.2)

Contrôle	Implémentation attendue
Rôle	Garde « `requireRole('admin')` » / « `requireRole('employee')` » / « authentifié ».
Propriété (salarié)	Le code salarié est **toujours** déduit du jeton (`/me/…`) et jamais pris dans le corps de la requête pour un salarié. Un salarié qui demande la ressource d'un autre reçoit `403/404`.
Salarié actif	Vérifié à chaque requête d'un salarié (ou via invalidation des jetons à la désactivation).
Champs protégés	Le salarié ne peut jamais fournir : durée, salaire, cumul, source, `entry_by`, statut, `decidedBy`…
Messagerie	Salarié : accès aux groupes (lecture de la liste pour pouvoir rejoindre) et aux conversations dont il est membre ; écriture interdite dans un groupe « annonce » ; seul l'admin supprime/vide/crée. Admin : accès à toutes les conversations.
Journal d'audit	Écrit par le serveur uniquement (jamais par le client) ; lecture admin.

16. API REST : endpoints principaux

Préfixe `/api/v1` ; JSON UTF-8 ; dates en ISO 8601 (`timestamptz`) et jours `YYYY-MM-DD` ; montants en entiers (Ar). Légende des accès : **A** = admin, **S** = salarié (ses données), ***** = tout utilisateur connecté, **P** = public. Chaque endpoint remplace une ou plusieurs fonctions de `src/services` (colonne de droite).

16.1 Authentification

Accès	Endpoint	Rôle / corps / réponse	Remplace
P	`POST /auth/login`	{ identifier, password } → { accessToken, user:{ id, role, employeeCode?, email? } } + cookie refresh. 401 générique si échec ; 403 si salarié désactivé.	`loginWithIdentifier`, `getAppUser`
P	`POST /auth/refresh`	Rotation du refresh token → nouveau jeton d'accès.	`onAuthStateChanged`
*	`POST /auth/logout`	Révoque le refresh token.	`logout`
*	`GET /auth/me`	Profil applicatif { id, role, employeeCode, email } (404/401 si salarié inactif). Appelé au chargement de l'application.	`getAppUser`
P	`POST /auth/register-admin`	{ email, password, inviteCode } → crée l'admin, renvoie une session (comme le login). 403 si code invalide ; 409 si email déjà pris.	`createAdminAccount`
A	`POST /auth/verify-password`	{ password } → 204 si correct (re-authentification avant action sensible).	`verifyAdminPassword`

16.2 Salariés et lieux de travail

Accès	Endpoint	Rôle / corps / réponse	Remplace
A	`GET /employees`	Liste triée par nom, avec `departmentIds` ; filtres optionnels `active`, `q`.	`listEmployees`
A / S	`GET /employees/:code` · `GET /me/employee`	Fiche (le salarié n'obtient que la sienne).	`getEmployee`
A	`POST /employees`	{ employeeCode, firstName, lastName, email?, phone?, position?, hireDate?, address?, baseDailySalary, departmentIds[≥1], password(≥6) } → crée fiche + compte dans **une seule transaction**. 409 si code existant.	`createEmployee`, `createEmployeeAccount`
A	`PATCH /employees/:code`	Mise à jour des champs modifiables (pas `employeeCode`, **pas** `active`, cf. §12 n°4). `departmentIds` remplace l'ensemble.	`updateEmployee`
A	`PATCH /employees/:code/active`	{ active } — désactiver/réactiver.	`setEmployeeActive`

A	`DELETE /employees/:code`	Supprime fiche + compte (règle d'historique, cf. §14.3). 409 si historique et suppression refusée.	`deleteEmployee`
A	`PUT /employees/:code/password`	{ newPassword(≥6) } — sans ancien mot de passe ; révoque les sessions du salarié.	`changeEmployeePassword`
A	`POST /employees/:code/password-reset` optionnel	Email de réinitialisation, si un email et un service d'envoi existent.	`sendEmployeePasswordReset`
*	`GET /workplaces`	Liste (tous les connectés ; nécessaire à l'accueil salarié et aux formulaires).	`listWorkplaces`
A	`POST /workplaces` · `PATCH /workplaces/:id` · `DELETE /workplaces/:id`	CRUD ; { name, latitude, longitude, radiusMeters, geofencingEnabled }. La suppression retire aussi les affectations (cascade).	`create/update/deleteWorkplace`
*	`GET /settings` · A: `PUT /settings`	{ dailySalary, normalWorkMinutes, toleranceMinutes, normalEntryTime }. Valeurs par défaut si jamais enregistrées.	`getSettings`, `updateSettings`

16.3 Pointage, rémunération, paiements, rapports

Accès	Endpoint	Rôle / corps / réponse	Remplace
S	`POST /attendance/clock-in`	{ workplaceId, location:{ latitude, longitude, altitude?, accuracy, address? } }. Serveur : département parmi ceux du salarié ; pas de session ouverte ; Haversine ; si géofencing actif et hors rayon → **422 GPS_OUT_OF_RANGE** { distance, radiusMeters } sans rien enregistrer ; sinon crée la session (heure serveur, `entry_source=EMPLOYEE`). Réponse : { attendance, gpsCheck }.	`clockIn`
S	`POST /attendance/clock-out`	{ location }. Cible la session ouverte du salarié ; mêmes contrôles GPS sur **le même département** ; **transaction** : calcule durée et salaire (§6.1), met à jour le pointage, crée l'enregistrement de rémunération (cumul). Réponse : { workedMinutes, dailySalary, gpsCheck }. 409 si aucune session ouverte.	`clockOut`, `finalizeAttendance`
A	`POST /admin/attendance/clock-in`	{ employeeCode, workplaceId } — sans GPS, `entry_source=ADMIN`, `entry_by` = admin ; audit.	`adminClockIn`
A	`POST /admin/attendance/:id/clock-out`	Même calcul qu'une sortie, `exit_source=ADMIN`, `exit_by` ; 409 si déjà terminé ; audit. (Décider si l'on autorise aussi les dates passées, cf. §12 n°8.)	`adminClockOut`

A	`GET /attendance?date=YYYY-MM-DD` (+ `from`, `to`, `employeeCode`)	Pointages d'un jour (ou période), avec nom du salarié joint.	`listAttendanceForDate`, `listAllAttendance`
S	`GET /me/attendance/today` · `GET /me/attendance?limit=60`	Sessions du jour (ordre chronologique) et historique du salarié (récent → ancien).	`listEmployeeSessions`, `getTodayAttendance`, `listAttendanceHistory`
S / A	`GET /me/balance` · `GET /employees/:code/balance`	{ earned, paid, due } calculés par le serveur.	`getCumulativeBalance`, `getTotalPaid`
A	`GET /salary/summary`	Une ligne par salarié { employee, earned, paid, due } — remplace les N+1 du Dashboard et de Rémunérations ; total global.	boucles `getCumulativeBalance` + `listPayments`
S	`GET /me/salary`	{ records[] (date, minutes, salaire, cumul), earned, paid, due, totalMinutes } ; tri par date de création décroissante.	`listSalaryHistory`
A	`POST /payments`	{ employeeCode, amount, note? } → 201 ; **422 AMOUNT_EXCEEDS_DUE** si > reste ; date = jour courant ; audit `PAYMENT_CREATED`.	`createPayment`
A / S	`GET /payments?employeeCode=` · `GET /me/payments`	Liste, plus récent d'abord.	`listPayments`
A	`GET /reports/salary?from=&to=&format=json|csv`	Agrégat par salarié : { employeeCode, name, days (jours distincts), minutes, total } trié par total décroissant + total général ; `format=csv` reproduit le fichier actuel.	`listSalaryForRange` + agrégation de `Reports.tsx`
A	`GET /dashboard/today`	Compteurs (actifs, présents, absents, minutes), paie du jour, cash restant global, top 5, en poste, retards, absents.	calculs de `Dashboard.tsx`

16.4 Matériel et prêts

Accès	Endpoint	Rôle / corps / réponse	Remplace
*	`GET /equipment`	Inventaire trié par nom ; contient l'emprunteur actuel (nom) quand `EMPRUNTE`.	`subscribeEquipment`
A	`POST /equipment` · `PATCH /equipment/:id` · `DELETE /equipment/:id` · `PATCH /equipment/:id/out-of-service`	CRUD ; { name, category?, serialNumber?, description? } ; `out-of-service` { outOfService } ; 409 si matériel emprunté (modification de statut / suppression).	`createEquipment`, `updateEquipment`, `deleteEquipment`, `setOutOfService`
A / S	`GET /loans` (filtres `status`, `equipmentId`) · `GET /me/loans`	Tous les prêts (admin) ou ceux du salarié ; du plus récent au plus ancien ; `GET /loans/pending-count` pour le badge admin.	`subscribeLoans`, `subscribePendingCount`
S	`POST /loans`	{ equipmentId, note? } ; 409 si matériel non disponible ou demande déjà en attente.	`requestLoan`
S	`POST /loans/:id/cancel` · `POST /loans/:id/return-request`	Annuler (`DEMANDE→ANNULEE`) ; signaler le retour { note? } (`EN_COURS→RETOUR_DEMANDE`) ; uniquement sur ses propres prêts et dans le bon statut.	`cancelRequest`, `reportReturn`

A	`POST /loans/:id/approve` · `/refuse` { reason? } · `/confirm-return`	Transactions avec verrou sur matériel + prêt, mêmes messages d'erreur qu'au §5.5.	`approveLoan`, `refuseLoan`, `confirmReturn`

16.5 Messagerie et administration

Accès	Endpoint	Rôle / corps / réponse	Remplace
*	`GET /chats`	Admin : toutes. Salarié : ses conversations + tous les groupes ; avec `unreadCount` par conversation et total.	`subscribeChats`, comptage du ChatContext
A	`POST /chats/groups` · `POST /chats/direct`	{ name, announcement } · { employeeCode } (idempotent : renvoie la conversation existante).	`createGroup`, `openDirectChat`
S	`POST /chats/:id/join` · `POST /chats/:id/leave`	Groupes seulement.	`joinGroup`, `leaveGroup`
*	`GET /chats/:id/messages?limit=300&before=`	Messages (avec réactions agrégées `{emoji: userIds[]}` et réponse citée), du plus ancien au plus récent ; pagination.	`subscribeMessages`
*	`POST /chats/:id/messages`	{ text(1–2000), replyToId? } ; l'expéditeur (nom, rôle) vient du jeton ; refus (403) d'un salarié dans un groupe « annonce » ; met à jour l'aperçu du chat ; émet l'événement temps réel.	`sendMessage`
*	`PUT /chats/:id/messages/:mid/reactions/:emoji` (bascule)	Ajoute ou retire la réaction de l'utilisateur.	`toggleReaction`
A	`DELETE /chats/:id/messages/:mid` · `POST /chats/:id/clear` · `DELETE /chats/:id`	Supprimer un message / vider (messages + lectures, aperçu remis à zéro) / supprimer la conversation.	`deleteMessage`, `clearConversation`, `deleteChat`
*	`PUT /chats/:id/read` · `GET /chats/:id/reads`	Marque la conversation comme vue (« Vu » et non-lus) / liste des accusés.	`markSeen`, `subscribeReads`, `markRead` (localStorage)
A	`POST /admin/reset-data`	{ password } ; supprime les mêmes données qu'aujourd'hui (§4.3 J) en **une transaction**, journalise l'opération.	`resetAllData`
A	`GET /audit-logs` optionnel	Consultation du journal (aucune page actuelle).	`logAudit` (écriture)

17. Temps réel (messagerie, matériel)

Aujourd'hui, `onSnapshot` met à jour l'écran instantanément pour : conversations, messages, accusés « Vu », inventaire du matériel, prêts, compteur de demandes. Le nouveau backend doit offrir un équivalent.

- **Canal** : WebSocket `/api/v1/ws` (authentifié par le jeton d'accès) ou SSE. Après connexion, le serveur n'envoie à chaque client que les événements qu'il a le droit de voir (mêmes règles d'accès que l'API).
- **Événements suggérés** : `chat.updated` (création, aperçu, membres), `message.created`, `message.deleted`, `message.reactions`, `chat.cleared`, `chat.deleted`, `chat.read`, `equipment.updated`, `loan.updated`, `loans.pending` (compteur).
- **Stratégie côté client** : à la connexion et à la reconnexion, recharger l'état par l'API REST (`GET`), puis appliquer les événements. En repli, un *polling* (ex. toutes les 5–10 s pour la messagerie et le matériel) reste acceptable si le WebSocket n'est pas retenu.
- **Plusieurs instances du serveur** : propager les événements via `LISTEN/NOTIFY` PostgreSQL ou Redis pub/sub.
- **Notifications locales Android** : continuent d'être déclenchées côté client à la réception d'un événement `message.created` quand la page est masquée (logique de `ChatContext` inchangée). Un vrai push serveur est une évolution distincte.

18. Validations et gestion des erreurs

18.1 Format d'erreur

```
HTTP 4xx/5xx   { "error": { "code": "GPS_OUT_OF_RANGE", "message": "Vous êtes actuellement à 142 mètres…", "details": { "distance": 142, "radiusMeters": 100 } } }
```

Le `message` est en **français**, directement affichable : le frontend l'utilise tel quel dans ses toasts (comme il le fait aujourd'hui avec `e.message`). Les messages existants à conserver mot pour mot sont repris au §5 et §6.

18.2 Codes d'erreur

HTTP	Code	Cas
400	`VALIDATION_ERROR`	Corps invalide (champ, type, borne, longueur) ; `details` liste les champs.
401	`INVALID_CREDENTIALS` / `UNAUTHENTICATED`	Mauvais identifiant/mot de passe ; jeton absent ou expiré.
403	`FORBIDDEN`, `ACCOUNT_DISABLED`, `INVALID_INVITE_CODE`, `ANNOUNCEMENT_READ_ONLY`	Rôle insuffisant, ressource d'autrui, salarié désactivé, code d'invitation faux, écriture dans une annonce.
404	`NOT_FOUND`	Ressource inconnue (ou non visible).
409	`ALREADY_CLOCKED_IN`, `NO_OPEN_SESSION`, `ALREADY_FINISHED`, `EMPLOYEE_CODE_TAKEN`, `EMAIL_TAKEN`, `EQUIPMENT_UNAVAILABLE`, `LOAN_ALREADY_PENDING`, `EQUIPMENT_BORROWED`, `LOAN_NOT_PENDING`, `LOAN_NOT_ACTIVE`, `HAS_HISTORY`	Conflits d'état (messages du §5, §6.3).
422	`GPS_OUT_OF_RANGE`, `AMOUNT_EXCEEDS_DUE`, `NO_DEPARTMENT_ASSIGNED`, `DEPARTMENT_NOT_ALLOWED`	Règles métier non satisfaites (rien n'est enregistré).
429	`RATE_LIMITED`	Trop de tentatives (login, création d'admin).
500	`INTERNAL_ERROR`	Erreur inattendue ; détail uniquement dans les journaux serveur.

18.3 Exigences transverses

- Toute opération multi-tables (création salarié + compte, sortie + rémunération, prêt + matériel, réinitialisation) s'exécute dans une **transaction SQL** ; en cas d'échec, rien n'est enregistré (corrige l'orphelin « compte Auth créé / fiche non créée » actuel).
- Horloge : toutes les heures d'entrée/sortie et la durée sont posées/calculées par le serveur ; le jour de travail est calculé dans le fuseau `Indian/Antananarivo`.
- Journalisation (`audit_logs`) côté serveur : au minimum paiements, pointages admin, création d'admin, changements de mot de passe, suppression/désactivation de salarié, réinitialisation, modification des paramètres (les types `EMPLOYEE_*` / `SETTINGS_UPDATED` existaient en TypeScript sans être émis).

- Sécurité HTTP : HTTPS, `helmet`, CORS restreint, limitation de débit, taille maximale des corps, requêtes paramétrées uniquement (pas de SQL concaténé), jamais de mot de passe ou de jeton dans les journaux.

19. Communication du frontend avec le nouveau backend

19.1 Principe : ne remplacer que la couche `services/`

Les pages React n'appellent jamais Firebase directement : elles passent par `src/services/*`. La stratégie la moins risquative consiste à **conserver les signatures des fonctions de service** et à réécrire leur contenu pour appeler l'API.

1. Créer `services/api.ts` : client `fetch` (URL de base depuis `import.meta.env.VITE_API_URL`), ajout automatique de `Authorization: Bearer`, **rafraîchissement automatique** du jeton sur 401 (une seule tentative), conversion des erreurs `{error:{code,message}}` en `Error(message)` (avec `code` attaché, ex. `GPS_OUT_OF_RANGE` → réutilisation de la classe `GpsOutOfRangeError` par `Home.tsx`).
2. Réécrire chaque fichier de `services/` selon la correspondance du §16 (colonne « Remplace »).
3. Remplacer les abonnements `subscribeX(cb)` par une souscription WebSocket qui renvoie la même fonction de désinscription ; la liste complète est rechargée à la connexion.

19.2 Adaptations nécessaires dans le code existant

Élément	Adaptation
`AuthContext`, `ProtectedRoute`, `useAuth`	Plus de `firebaseUser` (type `User` Firebase). Stocker { `user` (`AppUser`), `accessToken` }. `AppUser.uid` devient l'id utilisateur du backend ; les pages qui lisaient `firebaseUser.uid` (`Attendance`, `Equipment` admin, `ChatPage`, `ChatContext`) ou `appUser.uid` (`Salaries`) utilisent `appUser.id`. Au démarrage : `GET /auth/me` (ou refresh) à la place de `onAuthStateChanged`.
Types `Timestamp` Firestore	Remplacer par des chaînes ISO. Les appels `.toDate()` / `.toMillis()` (dans `Home, History, Dashboard, MapView, Attendance, Equipment, ChatConversation, ChatPage, ChatContext, utils/format.ts, services/attendance.ts, chat.ts, equipment.ts`) sont à remplacer par `new Date(iso)` ; ou fournir un petit adaptateur `toTs(iso)` exposant `toDate()/toMillis()` pour limiter les modifications.
Identifiants	Le code salarié reste la clé côté UI (`employee.employeeId`, `appUser.employeeId`). Les chats et prêts référencent des identifiants utilisateur (UUID) ; `emp.authUid` (utilisé par `ChatPage` pour ouvrir un message direct) est remplacé par `employeeCode`.
Champ `loginEmail` / `authUid`	Retirés du type `Employee` et du formulaire (§12 n°5). Le formulaire de création/édition envoie le mot de passe à `POST /employees`.
Section « mot de passe » de la modale salarié	Remplacer « mot de passe actuel + nouveau » par un seul champ « nouveau mot de passe » (`PUT /employees/:code/password`). Le lien « email de réinitialisation » devient optionnel.
Logique de calcul dans le navigateur	`clockIn/clockOut` n'envoient plus que { département, position } ; durée, salaire, cumul, GPS vérifié viennent du serveur. `utils/salaryCalculator.ts` reste utilisé **uniquement** pour l'aperçu « salaire estimé en direct » de l'accueil (`Home.tsx`) et doit rester strictement identique à la version serveur.
Agrégations	`Dashboard` et `Salaries` utilisent `GET /salary/summary` et `GET /dashboard/today` ; `Reports` utilise `GET /reports/salary` (l'export CSV peut rester généré côté client).
Réinitialisation	`Settings.tsx` appelle `POST /admin/reset-data` (le serveur revérifie le mot de passe).

Non-lus de messagerie	Remplacer le `localStorage["chat_read_{uid}"]` et `getCountFromServer` par `unreadCount` fourni par `GET /chats` et `PUT /chats/:id/read`.
Configuration	`.env` : `VITE_API_URL` (Vercel + build Capacitor). Le CORS du backend doit autoriser l'origine `https://suivie-fortico.vercel.app`.

20. Éléments Firebase à supprimer ou remplacer

Élément	Action	Remplacement / remarque
`package.json` : dépendance `firebase` (^10.13.0)	Supprimer	Aucun import `firebase/*` ne doit rester (`grep -r "firebase" src` doit être vide).
`src/services/firebase.ts` (config en dur, Auth, Firestore, Analytics)	Supprimer	Remplacé par `services/api.ts`.
`src/services/auth.ts`	Réécrire	Login, logout, `register-admin`, `me`, `refresh`. Disparaissent : instances « secondary », `createEmployeeAccount`, `changeEmployeePassword` (avec ancien mot de passe), `sendEmployeePasswordReset`, `getAppUser` via `users`.
`services/employees, workplaces, attendance, salary, payments, settings, equipment, chat, audit, reset`	Réécrire	Appels API (§16). Supprimer `runTransaction`, `writeBatch`, `serverTimestamp`, `arrayUnion/Remove`, `onSnapshot`, `getCountFromServer`…
`contexts/AuthContext.tsx`, `ChatContext.tsx`, `routes/ProtectedRoute.tsx`	Adapter	Types et sources de données (§19.2).
`firestore.rules`	Supprimer	Règles portées dans les gardes/contrôles du backend (§8 → §15).
`firestore.indexes.json`, `firebase.json`, `.firebaserc`	Supprimer	Remplacés par les index SQL (§14) et la configuration de déploiement du backend.
Collection `admin_invite/main`	Remplacer	Table `admin_invites` ou variable `ADMIN_SIGNUP_CODE`.
Firebase Analytics	Supprimer	Aucune donnée analytique exploitée dans le code.
Projet Firebase `pointage-c41b0`	Désactiver après recette	Conserver un **export complet** (Firestore + Auth) avant toute suppression ; ne supprimer le projet qu'après validation en production.
Textes de l'interface mentionnant Firebase	Mettre à jour	« Email de connexion Firebase » (Employees : formulaire, fiche, messages d'erreur `auth/…`), README (sections « Déploiement des règles Firestore », « Créer le premier compte administrateur », « Créer un salarié »).
`.gitignore`, README	Mettre à jour	Documenter `VITE_API_URL` et le démarrage du backend.

> Ne **pas** supprimer : Capacitor (`android/`, `capacitor.config.ts`), Leaflet, la logique de notifications locales (`services/notify.ts`), Vercel (`vercel.json`). Ils ne dépendent pas de Firebase.

21. Migration des données, plan de travail et recette

21.1 Correspondance Firestore → PostgreSQL

Source Firestore	Cible PostgreSQL	Transformations
Firebase Auth + `users`	`users`	Export des comptes (`firebase auth:export`). Les mots de passe Firebase sont hachés en **scrypt modifié** : soit les vérifier avec le paramétrage exporté (clé de hachage du projet) à la première connexion puis les re-hacher en argon2id, soit **réinitialiser tous les mots de passe** (les salariés utilisent des codes PIN fixés par l'admin : communication à prévoir). Table de correspondance `uid Firebase → uuid` conservée pendant la migration.
`employees`	`employees` + `employee_workplaces`	ID = `employeeId` ; `departmentIds[]` → lignes de liaison (ignorer les ids inexistants) ; `hireDate` texte → `date` (NULL si invalide) ; `loginEmail` abandonné ; `authUid` → lien via `users.employee_code`.
`workplaces`	`workplaces`	Conserver les **ids** d'origine dans une table de correspondance (les pointages y font référence). Coordonnées texte → nombres ; corriger/rejeter celles qui sont invalides.
`attendance`	`attendance`	`entryTime/exitTime` Timestamp → `timestamptz` ; `entryLocation/exitLocation` → colonnes ; `*GpsCheck` → `jsonb` ; `entryBy/exitBy` (uid) → uuid via la table de correspondance. Détecter les sessions « EN_COURS » multiples par salarié (violation de l'index unique) et les traiter (clôture manuelle avant import).
`salary_records`	`salary_records`	Conserver `cumulativeSalary` tel quel ; vérifier la cohérence : pour chaque salarié, le dernier cumul = Σ `dailySalary` (contrôle d'intégrité à rapporter, pas à « corriger » silencieusement).
`payments`, `settings/general`	`payments`, `settings`	`date` ("YYYY-MM-DD") → `payment_date` ; `createdBy` → uuid.
`equipment`, `equipmentLoans`	`equipment`, `equipment_loans`	Les champs dénormalisés `current*` de `equipment` sont abandonnés (dérivés des prêts actifs). Prêts dont le matériel n'existe plus → `equipment_id` NULL (noms copiés conservés).
`chats` (+ `messages`, `reads`)	`chats`, `chat_members`, `chat_messages`, `message_reactions`, `chat_reads`	`memberUids[]` → lignes ; `reactions {emoji: uid[]}` → lignes ; `dm_{uid}` → `direct_user_id` ; l'état « lu » stocké en `localStorage` ne peut pas être migré (compteurs repartent de zéro, à annoncer).
`audit_logs`	`audit_logs`	Import tel quel (`userId` → uuid si connu, sinon NULL).

21.2 Méthode

1. **Script d'import idempotent** (`scripts/import-firestore`) : lit un export Firestore (SDK Admin ou `gcloud firestore export`), transforme et insère par lots dans l'ordre des dépendances (settings → workplaces → employees → users → liaisons → attendance → salary_records → payments → equipment → loans → chats…). Rapport final : comptes par table, lignes rejetées avec motif.

2. **Contrôles de réconciliation** : nombre de documents = nombre de lignes ; pour chaque salarié : total gagné, total payé, reste à payer identiques entre Firestore (calcul actuel) et PostgreSQL (API) ; nombre de pointages par mois ; échantillon de pointages comparé champ par champ.
3. **Répétition à blanc** sur une copie de production, mesure de la durée ; puis **fenêtre de bascule** : geler les écritures Firestore (maintenance), export, import final, tests de fumée, mise en ligne du frontend + backend, communication aux utilisateurs.
4. **Plan de repli** : conserver Firestore en lecture seule pendant une période de grâce ; ne couper Firebase qu'après validation.

21.3 Découpage proposé

Lot	Contenu	Livrable / critère de fin
0	Prise en main : lancer l'application actuelle, lire ce document et le code (`services/` en priorité), valider avec le responsable les décisions du §12.	Liste de décisions signée.
1	Socle backend : projet, configuration, migrations SQL (§14), CI, journalisation, gestion d'erreurs, autorisation.	Base créée, endpoint de santé, tests d'intégration sur la base.
2	Authentification + salariés + lieux + paramètres + création d'admin.	Connexion admin/salarié, CRUD complet, parité Login/Salariés/Lieux/Paramètres.
3	Pointage (GPS, salaire, cumul, admin à distance), historique, carte, présences.	Tests unitaires du calcul (§6.1) et du GPS ; parité Accueil/Historique/Présences/Carte.
4	Rémunérations, paiements, dashboard, rapports (+ CSV).	Parité Rémunérations/Salaire/Dashboard/Rapports ; réconciliation des soldes.
5	Matériel et prêts (transactions).	Scénarios de concurrence (deux demandes simultanées) validés.
6	Messagerie + temps réel + non-lus serveur.	Parité Discussions admin/salarié (groupes, annonces, directs, réactions, « Vu »).
7	Réécriture de `services/` côté frontend, adaptation des contextes, retrait de Firebase (§19–§20).	`grep firebase` vide ; `npm run build` sans erreur.
8	Migration des données, recette, bascule, désactivation de Firebase.	Rapport de réconciliation validé ; mise en production.

21.4 Recette (scénarios minimaux à valider)

1. **Pointage nominal** : entrée dans le rayon, sortie dans le rayon, durée et salaire corrects, solde cumulé mis à jour.
2. **Hors rayon** (entrée puis sortie) : refus avec le message de distance, aucune écriture ; avec géofencing désactivé : accepté.
3. **Calcul** : 0 min, 4 h (= 10 000 Ar pour 20 000/8 h), 8 h, 10 h (plafond), deux sessions le même jour (2 h + 7 h → plafond appliqué au total), session à cheval sur minuit.
4. Double entrée refusée ; sortie sans entrée refusée ; pointage admin à distance (entrée + arrêt) avec marqueur « Admin » ; deux requêtes de sortie simultanées (un seul enregistrement de rémunération).
5. Paiement supérieur au reste refusé ; deux paiements simultanés ne dépassant pas ensemble le reste.
6. Matériel : demande → approbation → retour signalé → retour confirmé ; refus ; annulation ; deux approbations simultanées du même matériel (une seule réussit).
7. Messagerie : groupe, annonce (salarié en lecture seule), message direct, réponse, réaction, « Vu », non-lus, vider, supprimer ; notification locale Android en arrière-plan.

8. **Sécurité** : un salarié ne lit/modifie jamais les données d'un autre ; un salarié ne peut appeler aucun endpoint admin ; salarié désactivé = accès refusé ; jeton expiré = rafraîchissement transparent ; essais de login limités.
9. Réinitialisation des données (mot de passe requis) ; création d'admin avec bon/mauvais code.
10. Régression visuelle : toutes les pages du §4 sur mobile (Android WebView) et ordinateur, thème clair et sombre.

22. Points importants à conserver lors de la migration

1. **Les règles de rémunération** du §6.1 à l'identique (plafond journalier, proportionnalité, arrondi à l'Ar, salaire individuel + durée normale globale, plafond sur le **total du jour**, différence f(total) − f(avant)).
2. **Le géofencing** : Haversine (R = 6 371 000 m), contrôle à l'entrée *et* à la sortie, même département, drapeau `geofencingEnabled`, enregistrement du contrôle même quand il n'est pas bloquant, rayon par défaut 100 m, message de refus exact.
3. **Plusieurs sessions par jour** mais **jamais deux ouvertes** ; pointage admin à distance sans GPS (traçabilité `source` + auteur).
4. **Le code salarié** (`EMP-001`) comme identifiant de connexion et clé métier immuable.
5. **Le « jour » de travail en heure de Madagascar (UTC+3)** ; ne jamais dériver la date d'un `toISOString()` UTC (bug historique évité dans `utils/date.ts`).
6. **Reste à payer = gagné − payé**, un paiement ne peut pas le dépasser ; **historique des paiements non modifiable**.
7. **Cycle de prêt de matériel** à 6 statuts, attribution et retour **atomiques**, noms copiés dans l'historique, impossibilité de supprimer/mettre hors service un matériel emprunté.
8. **Messagerie** : groupes + annonces (lecture seule salarié) + message direct unique par salarié, réponse citée, 6 réactions rapides, « Vu » limité à mon dernier message, 2000 caractères, vidage/suppression réservés à l'admin, notifications locales en arrière-plan.
9. **Un salarié désactivé ou supprimé perd tout accès** immédiatement, mais ses données historiques restent consultables par l'admin (désactivation).
10. **Création d'admin depuis le login** avec code d'invitation ; **réinitialisation des données** avec re-saisie du mot de passe et double confirmation.
11. **Interface en français** : libellés, messages d'erreur et formats (`fr-FR`, « 12 500 Ar ») inchangés ; thème clair/sombre ; interface salarié pensée mobile (barre d'onglets en bas) ; application Android via Capacitor chargeant l'URL distante.
12. **Export CSV des rapports** : mêmes colonnes et nom de fichier.
13. **Pas de perte de données financières** : réconciliation par salarié obligatoire avant la bascule (§21.2).

23. Annexes

A. Glossaire

Terme	Définition
Département / Lieu de travail	Une ligne de `workplaces` : point GPS + rayon. « Département » et « lieu » désignent la même chose dans l'interface.
Session	Un pointage (une entrée et sa sortie). Plusieurs par jour possibles.
Géofencing	Restriction du pointage à un rayon autour d'un lieu.
Solde cumulé / « gagné »	Somme des salaires de toutes les sessions terminées (cumul du dernier `salary_record`).
Reste à payer / « Cash restant »	Gagné − Σ des paiements.
Ar (Ariary)	Monnaie de Madagascar ; montants entiers.
Pointage « Admin »	Entrée/sortie déclenchée à distance par un administrateur, sans position GPS.
Annonce	Groupe de discussion où seul l'admin peut écrire.

B. Statuts et énumérations

Énumération	Valeurs
Rôle	`admin`, `employee`
Statut de pointage	`EN_COURS` (« En cours »), `TERMINEE` (« Terminée »)
Source de pointage	`EMPLOYEE`, `ADMIN`
Statut du matériel	`DISPONIBLE` (Disponible), `EMPRUNTE` (Emprunté), `HORS_SERVICE` (Hors service)
Statut de prêt	`DEMANDE` (Demande en attente), `REFUSEE` (Refusée), `ANNULEE` (Annulée), `EN_COURS` (Emprunté), `RETOUR_DEMANDE` (Retour à confirmer), `RETOURNE` (Retourné)
Type de conversation	`group`, `direct`
Réactions rapides	👍 ❤️ 😂 😮 😢 🙏
Valeurs par défaut des paramètres	salaire 8 h = 20 000 Ar ; durée normale 480 min ; heure d'entrée 08:00 ; tolérance 10 min ; rayon d'un lieu 100 m ; centre de carte par défaut : Toamasina (−18,15 ; 49,40).

C. Fichiers de référence à lire en priorité

Fichier	Pourquoi
`src/services/attendance.ts`	Cœur métier : GPS, session, salaire, cumul, pointage admin.
`src/utils/salaryCalculator.ts`, `distance.ts`, `date.ts`	Formules à reproduire exactement côté serveur.
`src/services/equipment.ts`	Cycle de prêt et transactions.
`src/services/chat.ts`, `contexts/ChatContext.tsx`, `components/chat/*`	Messagerie, non-lus, « Vu », notifications.
`src/services/auth.ts`, `employees.ts`, `contexts/AuthContext.tsx`	Comptes, rôles, création de salarié/admin.
`firestore.rules`	Référence des contrôles d'accès à reproduire.

`src/pages/admin/Dashboard.tsx`, `Reports.tsx`, `Salaries.tsx`	Agrégations à déplacer côté SQL.

Document établi par analyse directe du code du dépôt au 1er octobre 2026. Les recommandations d'architecture (§13 à §21) sont des propositions ; les comportements décrits aux §3 à §12 reflètent le code actuel.